Белорусский государственный технологический университет
Факультет информационных технологий
Кафедра программной инженерии

ОТЧЕТ

по лабораторной работе № 5
по дисциплине «Операционные системы»
Диспетчеризация и планирование

Выполнил:
Студент 3 курса 9 группы
Антипов Алексей Романович
Преподаватель: Уласевич Н. И.

СОДЕРЖАНИЕ

Введение ... 3
1 Основы планирования и условия исследования ... 4
1.1 Приоритеты и маски родственности ... 4
1.2 Методика измерений ... 4
1.3 Среда выполнения ... 5
2 Исследование планирования в Windows ... 7
2.1 Информация о текущем исполнителе ... 7
2.2 Циклическая рабочая нагрузка ... 7
2.3 Два дочерних процесса ... 8
2.4 Два дочерних потока ... 14
2.5 Сопоставление результатов Windows ... 20
3 Исследование планирования в Linux ... 22
3.1 Информация о текущем исполнителе ... 22
3.2 Циклическая рабочая нагрузка ... 22
3.3 Два дочерних процесса ... 23
3.4 Два дочерних потока ... 27
3.5 Сопоставление процессов и потоков Linux ... 31
4 Моделирование алгоритмов планирования ... 32
4.1 Определение статистики ... 32
4.2 Алгоритм Shortest Job First ... 33
4.3 Алгоритм Round Robin ... 33
4.4 Вытесняющее приоритетное планирование ... 34
4.5 Выбор кванта и сравнение алгоритмов ... 34
4.6 Итоговое сопоставление ... 36
5 Многоуровневая очередь с обратной связью ... 37
5.1 Параметры и правила модели ... 37
5.2 Результаты воспроизводимого запуска ... 37
5.3 Учёт процессов по очередям ... 39
5.4 Анализ переходов и обслуживания ... 40
Заключение ... 41
Список использованных источников ... 42
Приложение А Структура программ и ключевые операции ... 43
Приложение Б Ответы на вопросы для самоконтроля ... 44

Введение

Цель работы — получить практические навыки управления планированием процессов и потоков в Windows и Linux и исследовать алгоритмы распределения процессорного времени на заданных наборах задач.

Работа включает определение приоритетов и масок родственности, запуск двух одинаковых процессов и потоков с различными приоритетами, фиксацию их прогресса при первом завершении и сопоставление результатов двух операционных систем.

Для достижения цели выполнены следующие задачи:

– разработаны приложения Lab-05a, Lab-05x, Lab-05b и Lab-05c для Windows и Linux;

– проведены опыты на доступном процессоре;

– реализованы SJF, Round Robin и вытесняющее приоритетное планирование;

– сопоставлена статистика при различных квантах времени;

– реализована многоуровневая очередь с обратной связью для 100 процессов.

Натурные опыты выполнены в Windows и Linux с одинаковой периодической задержкой. Для SJF, RR, CP и MQ построены модельные расписания и рассчитаны характеристики обслуживания без выполнения реальных пользовательских заданий.

1 Основы планирования и условия исследования

Планировщик выбирает готовый поток, которому предоставляется процессор. Диспетчер выполняет переход к выбранному потоку: сохраняет и восстанавливает контекст, переключает адресное пространство при необходимости и передаёт управление. Приоритет определяет порядок выбора среди готовых потоков, но не устраняет ожидание таймера или ввода-вывода.

1.1 Приоритеты и маски родственности

В Windows базовый приоритет потока определяется сочетанием класса процесса и относительного приоритета потока. Для класса Normal относительные уровни Lowest, Normal и Highest дают базовые значения 6, 8 и 10. Основные потоки процессов Below Normal и High с относительным уровнем Normal имеют базовые приоритеты 6 и 13. Текущий динамический приоритет может отличаться от базового вследствие повышений планировщика.

В Linux для обычного планирования используется nice от −20 до 19: меньшее число соответствует большему весу. Изменение nice не переводит поток в режим реального времени. В Linux nice является атрибутом отдельного потока; отрицательные значения обычно требуют соответствующих прав. Поле PRI утилиты ps и значение nice следует различать.

Маска родственности ограничивает набор логических процессоров, на которых разрешено выполнение. Ненулевой бит с номером k разрешает CPU k. Количество установленных битов определяет число доступных процессоров, а не число физических ядер. В опытах используются полная разрешённая маска и один её младший установленный бит.

1.2 Методика измерений

Каждый рабочий исполнитель выполняет 1 000 000 итераций. После каждой тысячи итераций он ожидает 200 мс и выводит номер итерации, PID, TID, сведения о приоритете и текущий CPU. Сумма запрошенных задержек равна 200 с. Дополнительные параметры количества итераций и задержки предназначены для проверки программы; в обязательной серии применяется режим 1 000 000 и 200 мс.

Дочерние исполнители стартуют после настройки приоритетов и общего сигнала запуска. При первом завершении сохраняется снимок двух атомарных счётчиков ITER1 и ITER2. Разность DELTA равна модулю их разности. Счётчик показывает число выполненных итераций; исполнитель может уже иметь значение 1 000 000 и ещё находиться в последней задержке.

Программа выводит WALL_SECONDS по монотонным часам и CLOCK_SECONDS по clock. В Microsoft CRT clock измеряет прошедшее

календарное время процесса; в Linux clock отражает процессорное время процесса. Для потокового опыта Linux это общее CPU-время процесса, а не отдельного потока. Эти показатели нельзя непосредственно считать одной и той же межплатформенной метрикой.

1.3 Среда выполнения

Исследование выполнено в гостевых Windows 10 Pro x64 версии 10.0.19045 и Ubuntu 26.04.1 LTS с ядром 7.0.0-31-generic. В обеих средах отображается AMD Ryzen 7 7735H with Radeon Graphics; доступность восьми логических CPU подтверждена масками приложений. Для Linux разрешены CPU 0–7.

Конфигурация гостевой Windows представлена на рисунке 1.1.

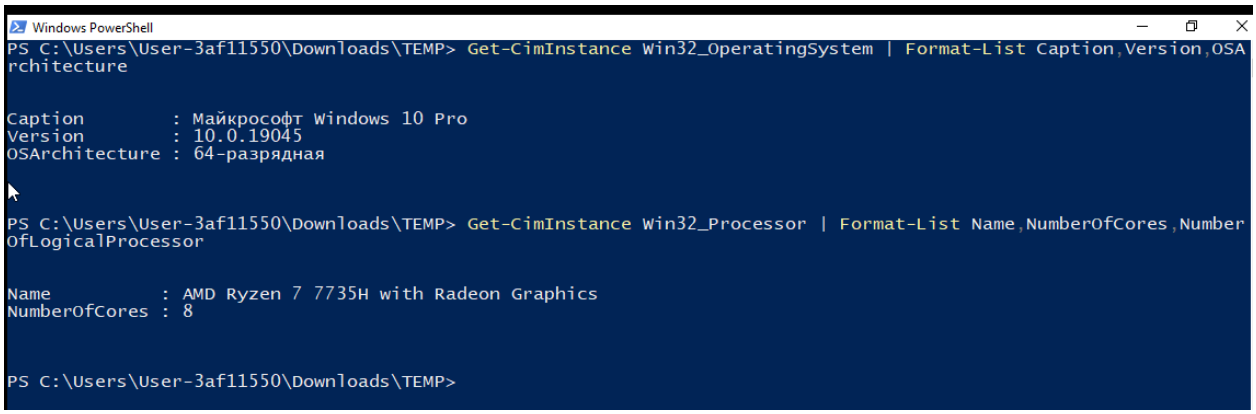


Рисунок 1.1 – Среда выполнения Windows

В Windows получена версия 10.0.19045 и 64-разрядная архитектура. Запрос Win32_Processor показывает восемь ядер; доступность восьми логических CPU для исследуемого процесса дополнительно подтверждается Lab-05a. Эти сведения задают конфигурацию гостевой системы, в которой получены последующие результаты; их следует учитывать при сопоставлении запусков.

Конфигурация Ubuntu и разрешённый набор CPU представлены на рисунке 1.2.

```
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04.1 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ uname -r
7.0.0-31-generic
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ lscpu | grep -E 'Model name|CPU\(s\)|Thread|Core|Socket'
CPU(s):                               8
On-line CPU(s) list:                  0-7
NUMA node0 CPU(s):                    0-7
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ python3 -c 'import os; print("Allowed CPU IDs:", sorted(os.sched_getaffinity(0)))'
Allowed CPU IDs: [0, 1, 2, 3, 4, 5, 6, 7]
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ LC_ALL=C lscpu | grep -E 'Model name|CPU\(s\)|Thread|Core|Socket'
CPU(s):                               8
On-line CPU(s) list:                  0-7
Model name:                           AMD Ryzen 7 7735H with Radeon Graphics
Thread(s) per core:                   1
Core(s) per socket:                   8
Socket(s):                            1
NUMA node0 CPU(s):                    0-7
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$
```

Рисунок 1.2 – Среда выполнения Linux

В Ubuntu разрешённый набор состоит из CPU 0–7. Полная маска равна 255, то есть 11111111_2, маска одного процессора — 1. В опытах с маской 1 используется CPU 0. Снимок показывает один аппаратный поток на ядро в конфигурации гостевой системы. При интерпретации опыта число доступных CPU определяется разрешённой маской процесса, а не только названием физического процессора.

2 Исследование планирования в Windows

Приложения Windows используют GetPriorityClass, GetThreadPriority, GetProcessAffinityMask, SetProcessAffinityMask и GetCurrentProcessorNumber. Lab-05b создаёт процессы через CreateProcess с отдельными консолями; Lab-05c запускает два рабочих потока в одном процессе. Числовые параметры класса процесса: Normal — 32, Below Normal — 16384, High — 128 [1, 3].

2.1 Информация о текущем исполнителе

Lab-05a выводит PID, TID, класс процесса, относительный и вычисленный базовый приоритет потока, двоичные маски процесса и системы, число разрешённых CPU и CPU, на котором выполнен запрос.

Результат контрольного запуска Lab-05a приведён на рисунке 2.1.

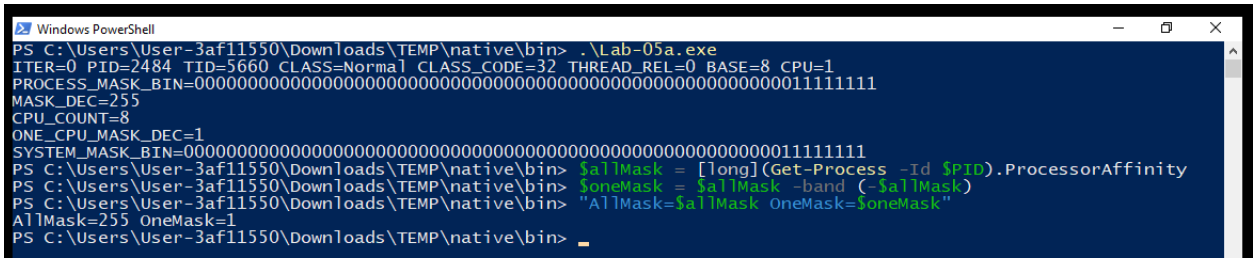


Рисунок 2.1 – Вывод Lab-05a в Windows

В контрольном запуске PID равен 2484, TID — 5660. Получены класс Normal (32), THREAD_REL = 0 и BASE = 8. Маски процесса и системы равны 255, CPU_COUNT = 8, текущий CPU = 1. GetThreadPriority возвращает относительный уровень; поле BASE содержит вычисленный базовый приоритет. Доступная маска задаёт исходное ограничение для следующих опытов. Текущий номер CPU характеризует лишь момент вызова функции.

2.2 Циклическая рабочая нагрузка

Lab-05x выполняет предусмотренный цикл без изменения приоритетов и маски. При запуске из Lab-05b эти параметры наследуются от настроенного дочернего процесса. Вывод производится после задержки, номер CPU считывается в момент формирования строки.

Последние итерации и показатели времени работы Lab-05x представлены на рисунке 2.2.

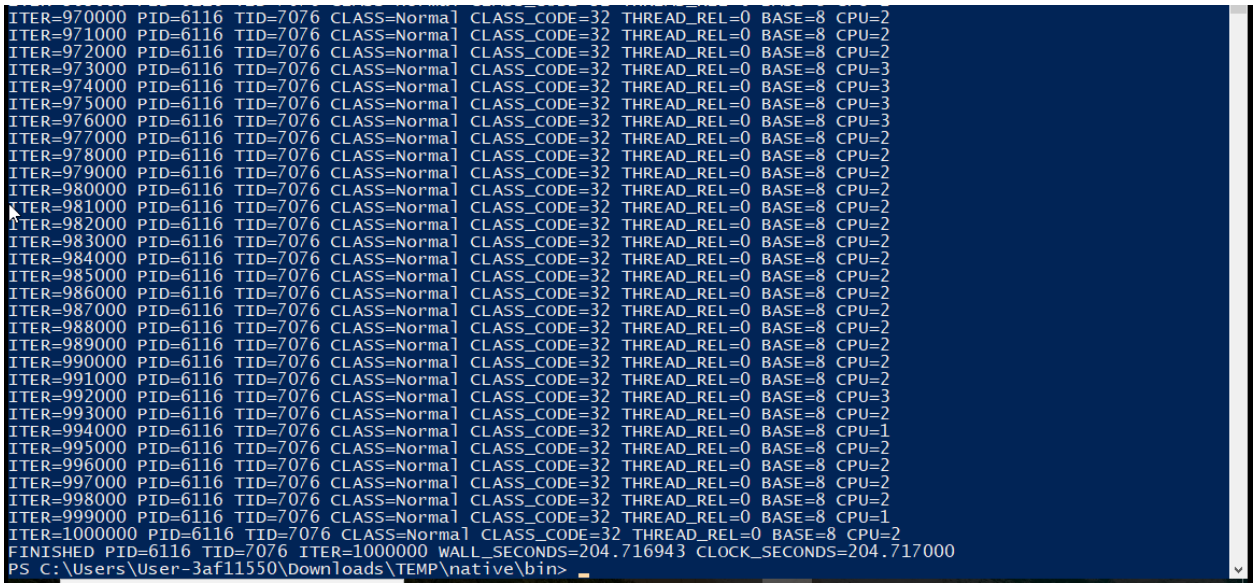


Рисунок 2.2 – Завершение Lab-05x в Windows

Процесс PID = 6116, TID = 7076 выполнил 1 000 000 итераций. WALL_SECONDS = 204,716943 с, CLOCK_SECONDS = 204,717000 с. Близость значений соответствует реализации clock в Microsoft CRT. В последних строках встречаются CPU 1, 2 и 3, что подтверждает миграции в пределах разрешённой маски. Регулярные задержки входят в полное время выполнения, поэтому продолжительность этого запуска нельзя трактовать как время непрерывного вычисления на CPU.

2.3 Два дочерних процесса

Lab-05b запускает два одинаковых дочерних процесса. В каждом варианте используется одинаковая нагрузка, а число итераций сохраняется при первом завершении.

2.3.1 Все доступные CPU и приоритеты Normal / Normal

Вариант 1: все доступные CPU, приоритеты Normal / Normal. Параметры командной строки для приоритетов: 32 32. Исследуются разные процессы, поэтому необходимо сопоставить оба PID.

Параметры процессов в Process Explorer представлены на рисунке 2.3.

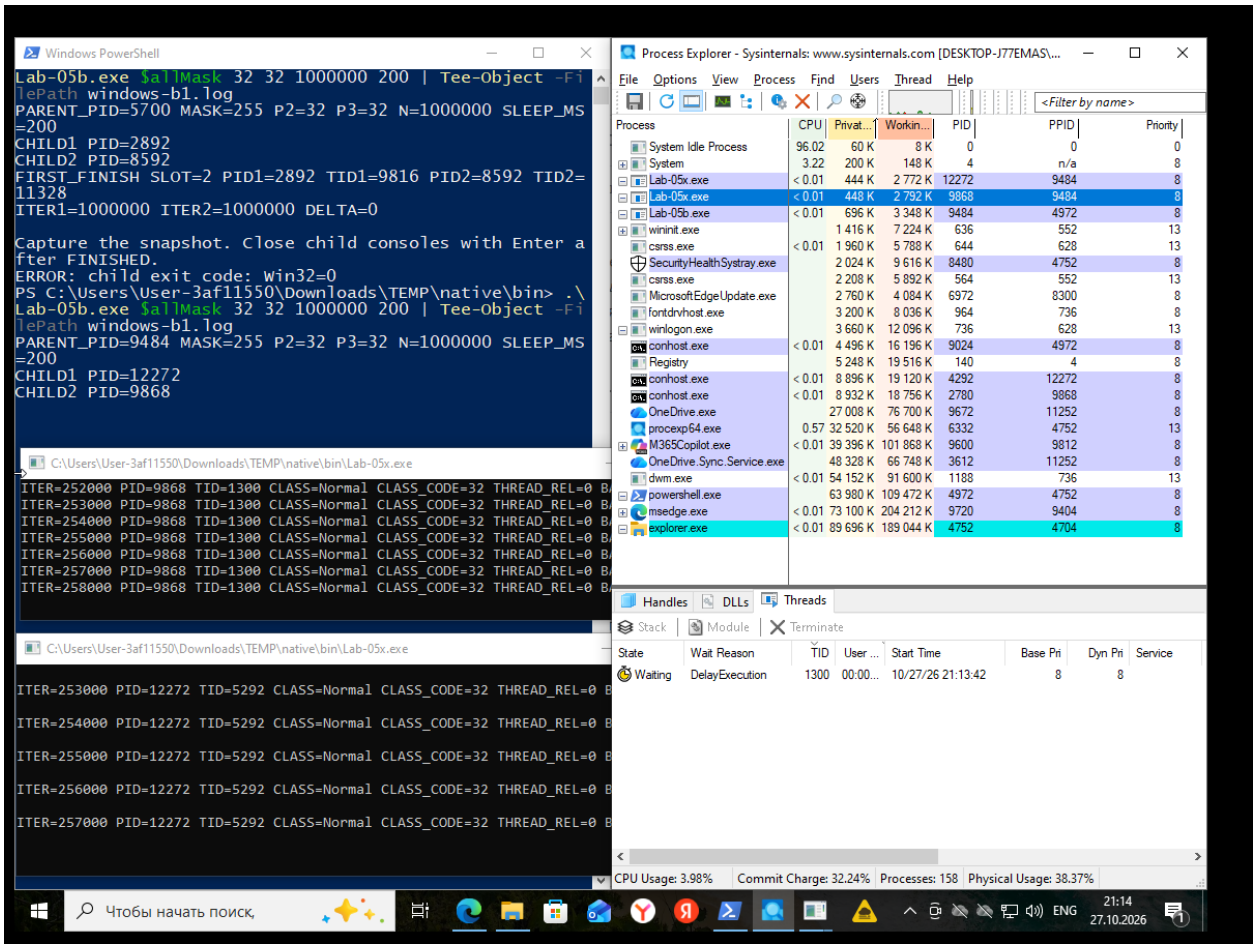


Рисунок 2.3 – Приоритеты и маска Lab-05b, вариант 1

Для текущего запуска родитель имеет PID = 9484, дочерние процессы — PID = 12272 и 9868. В Process Explorer оба дочерних процесса имеют приоритет 8. Их классы Normal и относительные уровни 0 соответствуют одинаковому базовому приоритету основных потоков. Совпадение настроек двух исполнителей задаёт контрольный вариант: различие прогресса в нём нельзя объяснить специально назначенным различием классов.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 2.4.

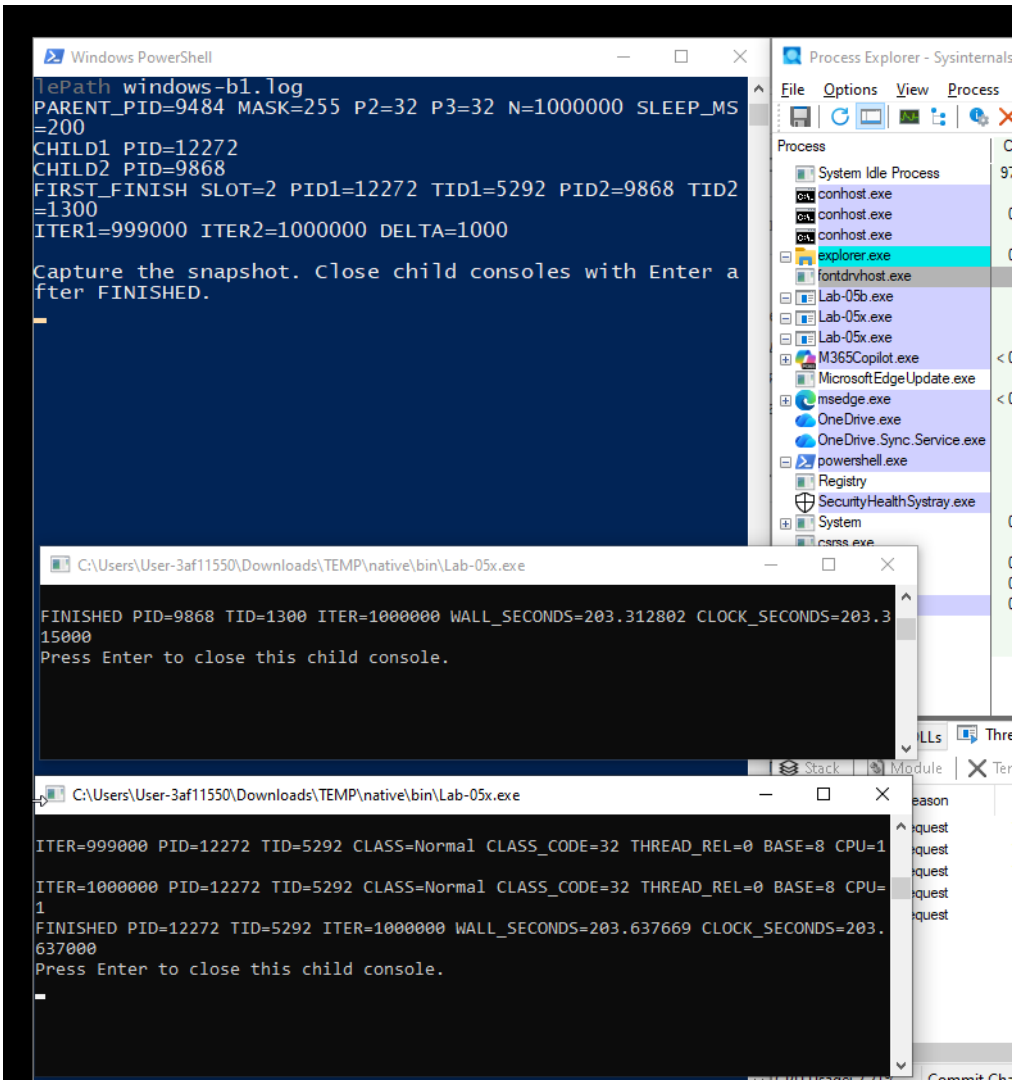


Рисунок 2.4 – Прогресс Lab-05b при первом завершении, вариант 1

Первым зарегистрировано завершение второго процесса: PID = 9868, TID = 1300, SLOT = 2. В этот момент ITER1 = 999000, ITER2 = 1000000, DELTA = 1000. Время второго процесса составило 203,312802 с, первого — 203,637669 с. При одинаковых приоритетах небольшое расхождение связано с конкретным порядком обслуживания и выводом; оно не указывает на устойчивое преимущество одного процесса. Снимок прогресса сохраняется один раз и затем не изменяется при дальнейшем выполнении первого процесса. Поэтому он характеризует именно момент первого завершения.

2.3.2 Все доступные CPU и приоритеты Below Normal / High

Вариант 2: все доступные CPU, приоритеты Below Normal / High. Параметры командной строки для приоритетов: 16384 128. Исследуются разные процессы, поэтому необходимо сопоставить оба PID.

Параметры процессов в Process Explorer представлены на рисунке 2.5.

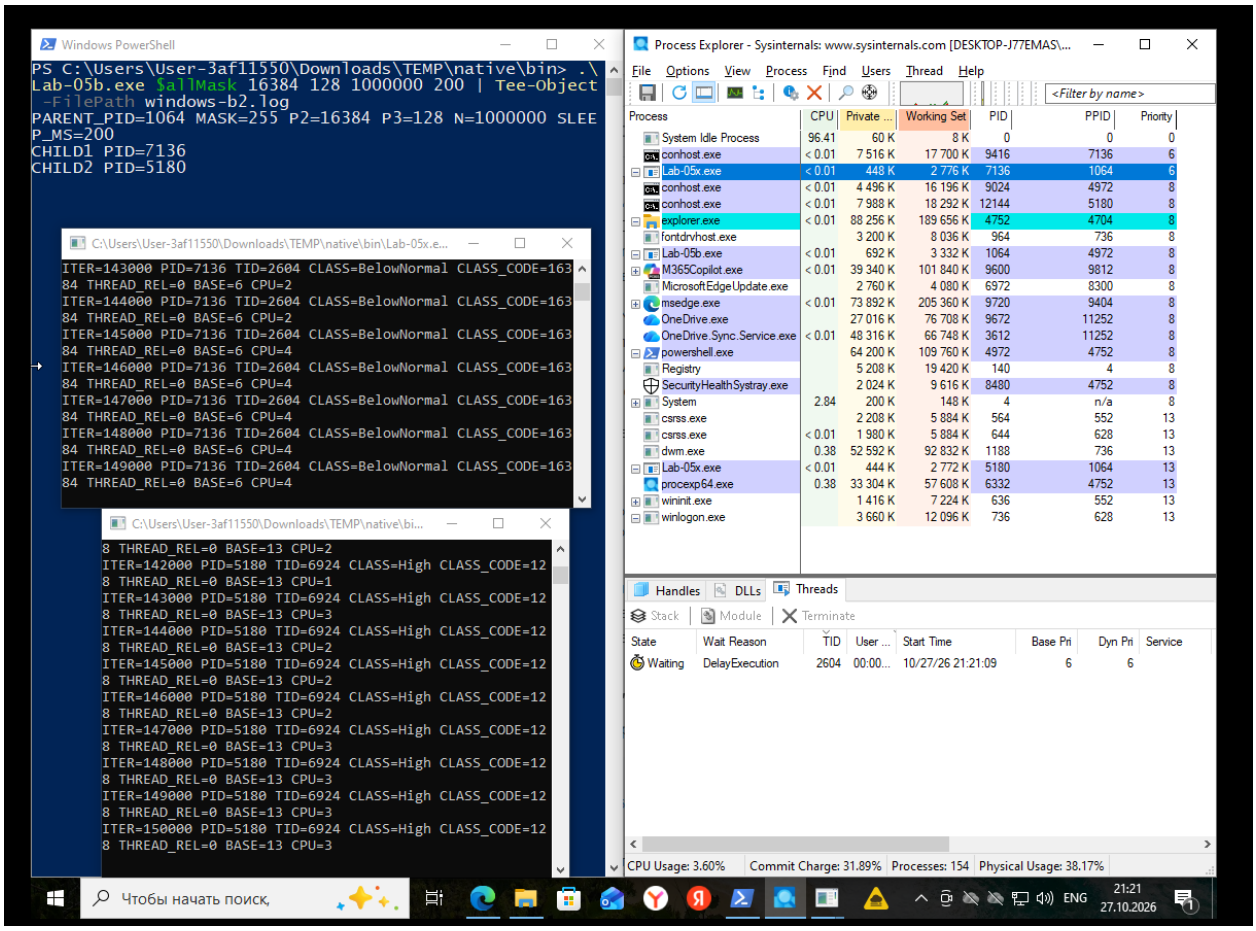


Рисунок 2.5 – Приоритеты и маска Lab-05b, вариант 2

Родительский PID = 1064, дочерние PID = 7136 и 5180. Для Below Normal показаны CLASS_CODE = 16384 и BASE = 6, для High — CLASS_CODE = 128 и BASE = 13. Значения согласуются с Process Explorer; в выводе видны разные номера разрешённых CPU. Проверка фактических классов отделяет корректность настройки от наблюдаемого эффекта приоритета. Разрешение нескольких CPU сохраняет возможность одновременного выполнения.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 2.6.

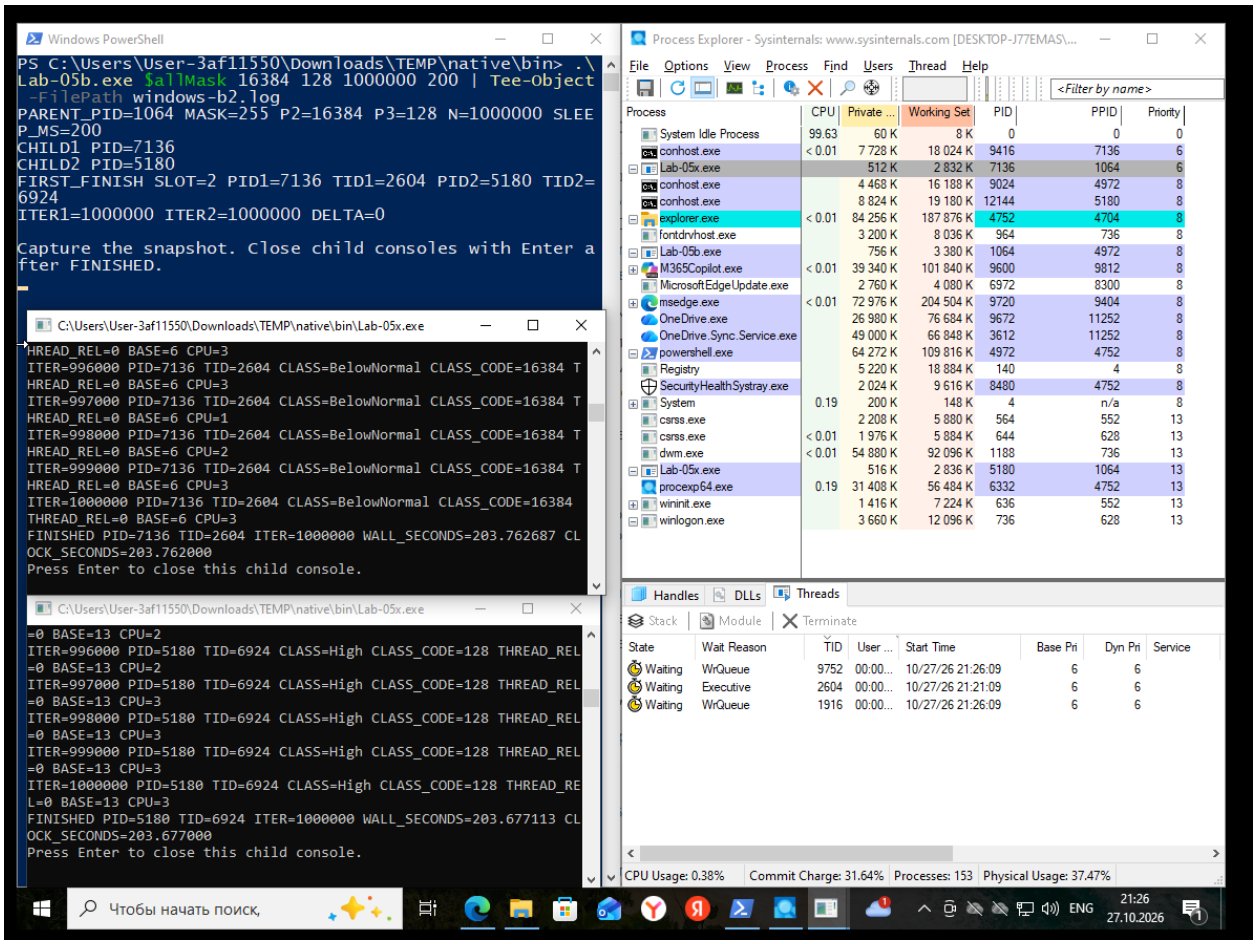


Рисунок 2.6 – Прогресс Lab-05b при первом завершении, вариант 2

Первым завершился второй процесс: PID = 5180, TID = 6924, SLOT = 2. Зарегистрированы ITER1 = 1000000, ITER2 = 1000000 и DELTA = 0. Времена High и Below Normal равны 203,677113 и 203,762687 с соответственно. Разные моменты завершения при равных счётчиках допустимы: последняя задержка выполняется после установки счётчика в 1 000 000. Счётчик обновляется до периодической задержки. Поэтому одинаковое число итераций не требует одновременного выхода из рабочих функций.

2.3.3 Один CPU и приоритеты Below Normal / High

Вариант 3: один CPU, приоритеты Below Normal / High. Параметры командной строки для приоритетов: 16384 128. Исследуются разные процессы, поэтому необходимо сопоставить оба PID.

Параметры процессов в Process Explorer представлены на рисунке 2.7.

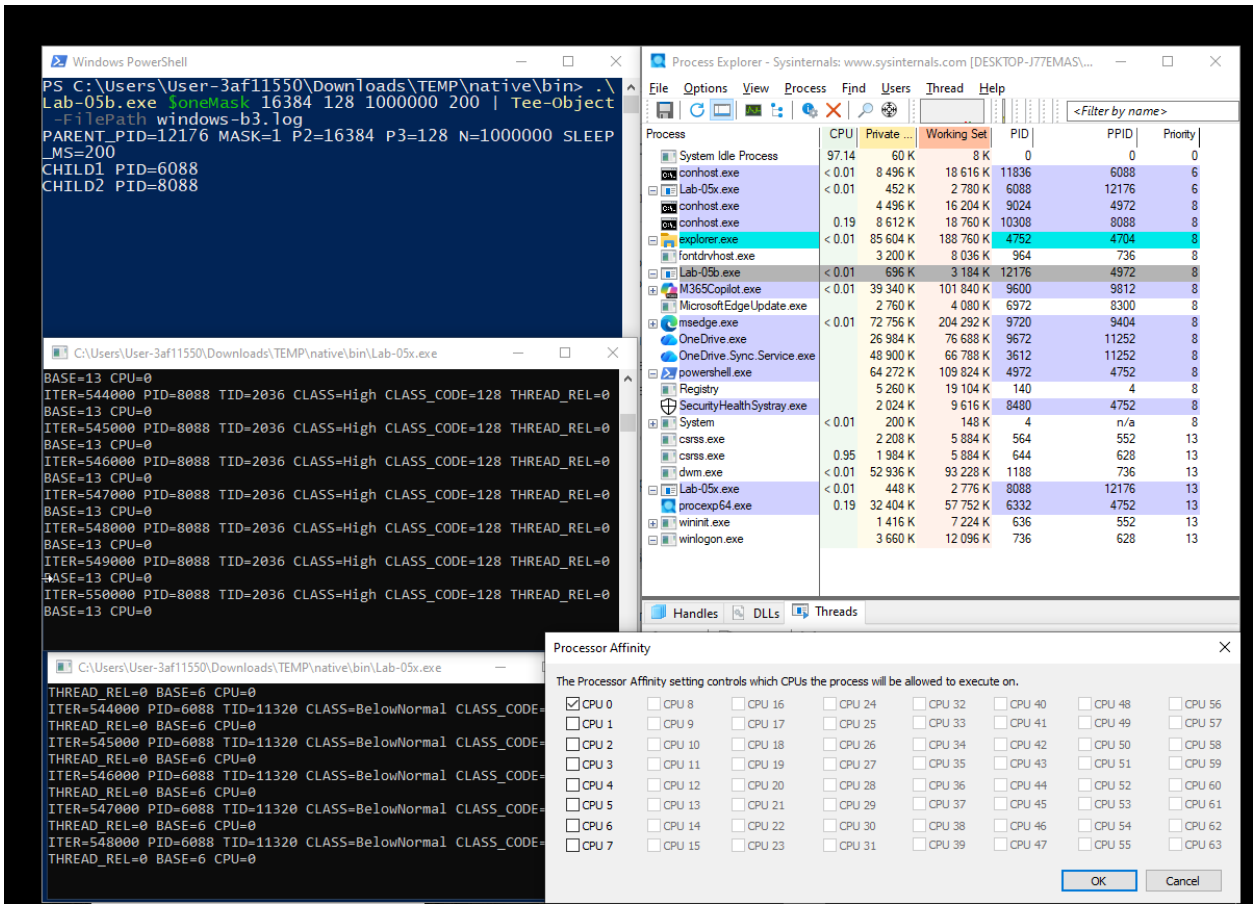


Рисунок 2.7 – Приоритеты и маска Lab-05b, вариант 3

Родительский PID = 12176, дочерние PID = 6088 и 8088. Маска 1 разрешает только CPU 0, что подтверждается окном Process Affinity и рабочими строками. Основные потоки Below Normal и High имеют базовые приоритеты 6 и 13 соответственно. Закрепление обоих процессов за одним CPU создаёт общий разрешённый ресурс выполнения. Ожидание таймера при этом продолжает освобождать CPU.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 2.8.

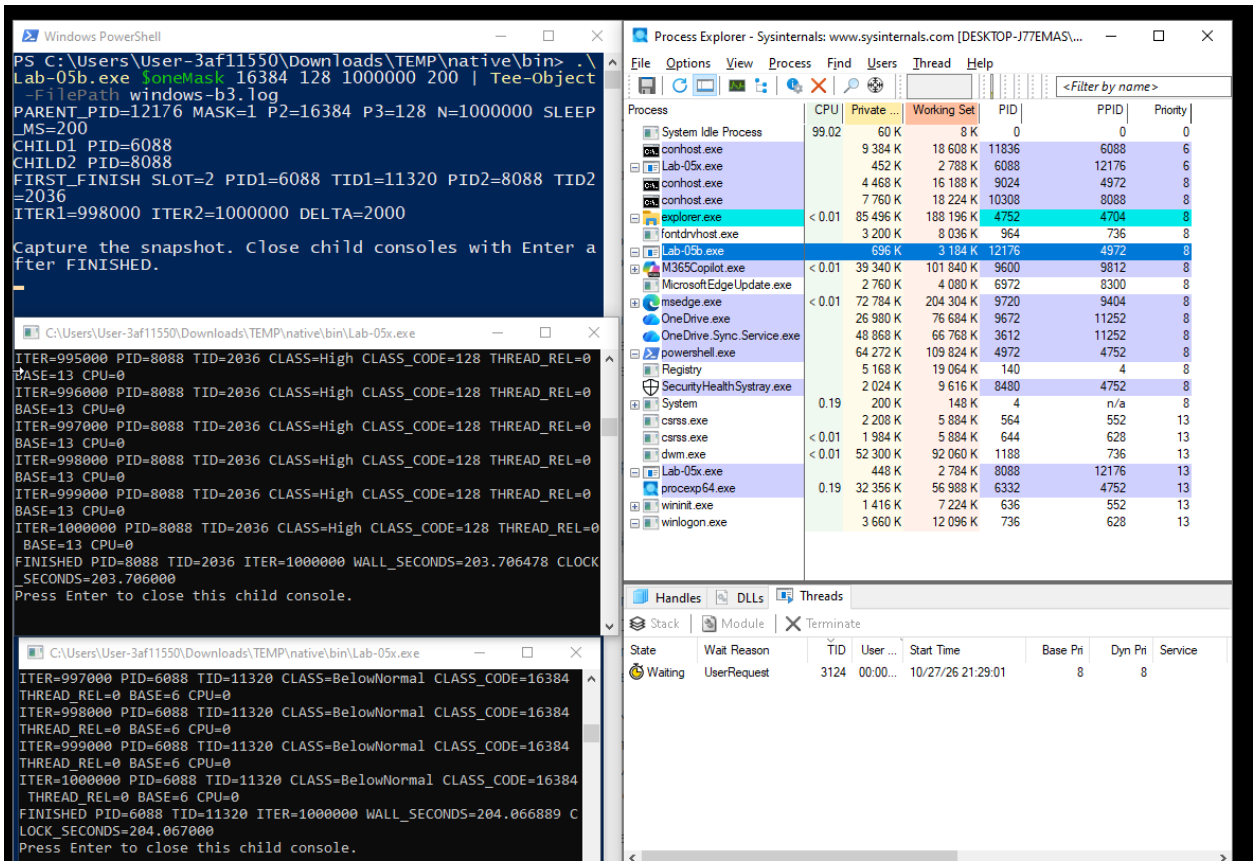


Рисунок 2.8 – Прогресс Lab-05b при первом завершении, вариант 3

Первым завершился процесс High: PID = 8088, TID = 2036, SLOT = 2. На сохранённом снимке ITER1 = 998000, ITER2 = 1000000 и DELTA = 2000. Время High составило 203,706478 с, Below Normal — 204,066889 с. В данном запуске высокоприоритетный процесс опередил второй на одном CPU; один опыт не определяет постоянную величину выигрыша. Сравнение с вариантом полной маски показывает изменение результата конкретного запуска. Для оценки устойчивости потребовалась бы серия повторных измерений.

2.4 Два дочерних потока

Lab-05c запускает два одинаковых рабочих потока. Для Windows процесс Lab-05c имеет класс Normal; относительные приоритеты задаются отдельно.

2.4.1 Все доступные CPU и приоритеты Normal / Normal

Вариант 1: все доступные CPU, приоритеты Normal / Normal. Параметры командной строки для приоритетов: 0 0. Оба исполнителя имеют один PID и разные TID.

Параметры потоков в Process Explorer представлены на рисунке 2.9.

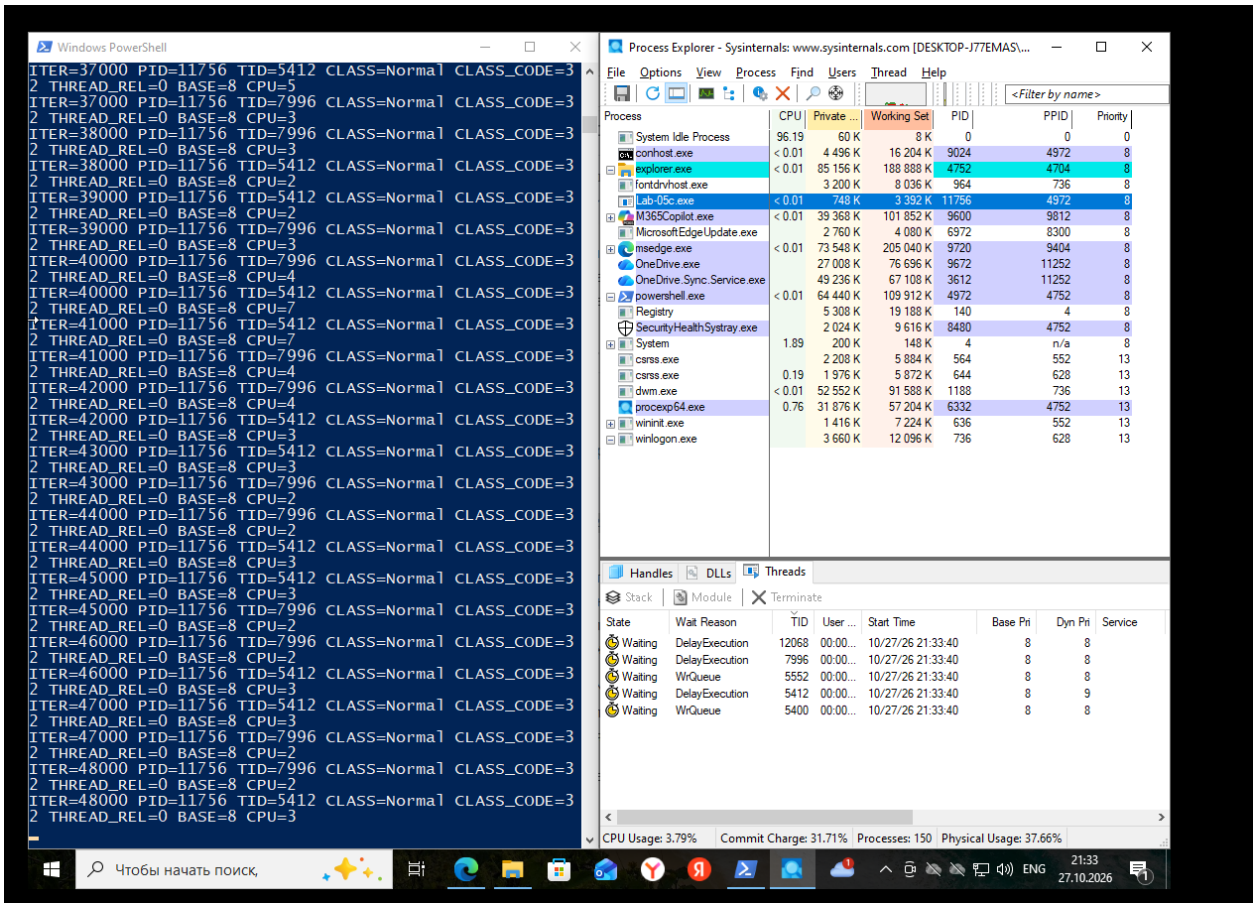


Рисунок 2.9 – Приоритеты и маска Lab-05c, вариант 1

Оба рабочих потока принадлежат PID = 11756 и имеют TID = 7996 и 5412. В Process Explorer для этих TID показан Base Pri = 8; рабочие строки содержат THREAD_REL = 0 и BASE = 8. Использование разных CPU соответствует полной маске 255. Различные TID позволяют отдельно проверять параметры рабочих потоков даже при общем процессе. Служебный главный поток не заменяет эти две записи.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 2.10.

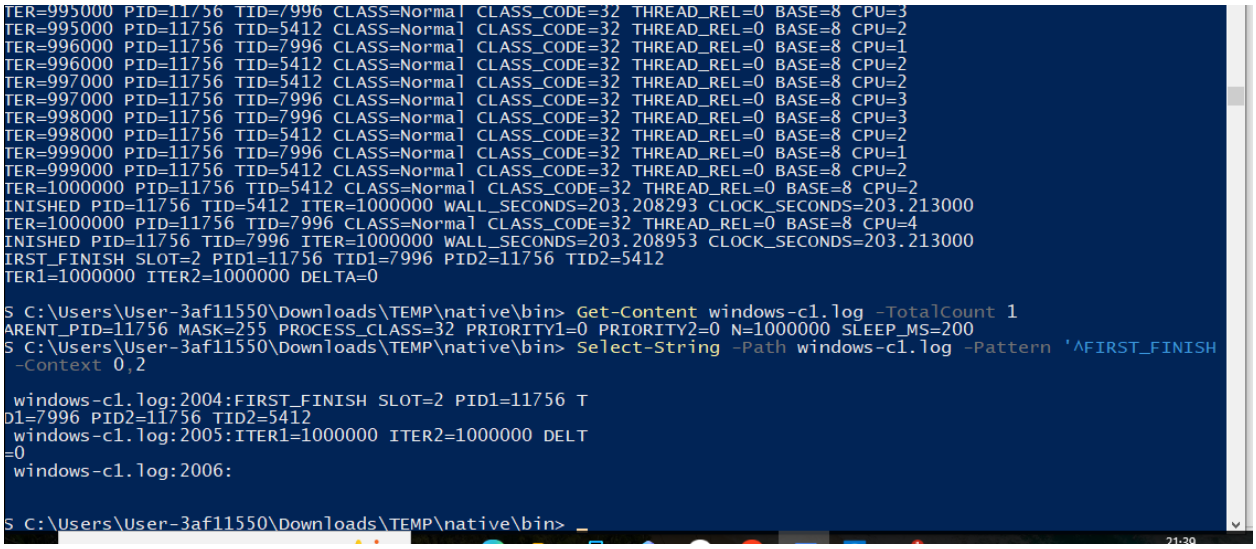


Рисунок 2.10 – Прогресс Lab-05c при первом завершении, вариант 1

В FIRST_FINISH зарегистрирован SLOT = 2: PID = 11756, TID = 5412. Оба счётчика равны 1000000, DELTA = 0. Время потока TID = 5412 составило 203,208293 с, TID = 7996 — 203,208953 с. Равные параметры и периодическое ожидание привели к почти совпавшему прогрессу. Зарегистрированный первый выход и разность итераций являются разными характеристиками. Порядок выхода при нулевой разности не означает отставания по объёму цикла.

2.4.2 Все доступные CPU и приоритеты Lowest / Highest

Вариант 2: все доступные CPU, приоритеты Lowest / Highest. Параметры командной строки для приоритетов: -2 2. Оба исполнителя имеют один PID и разные TID.

Параметры потоков в Process Explorer представлены на рисунке 2.11.

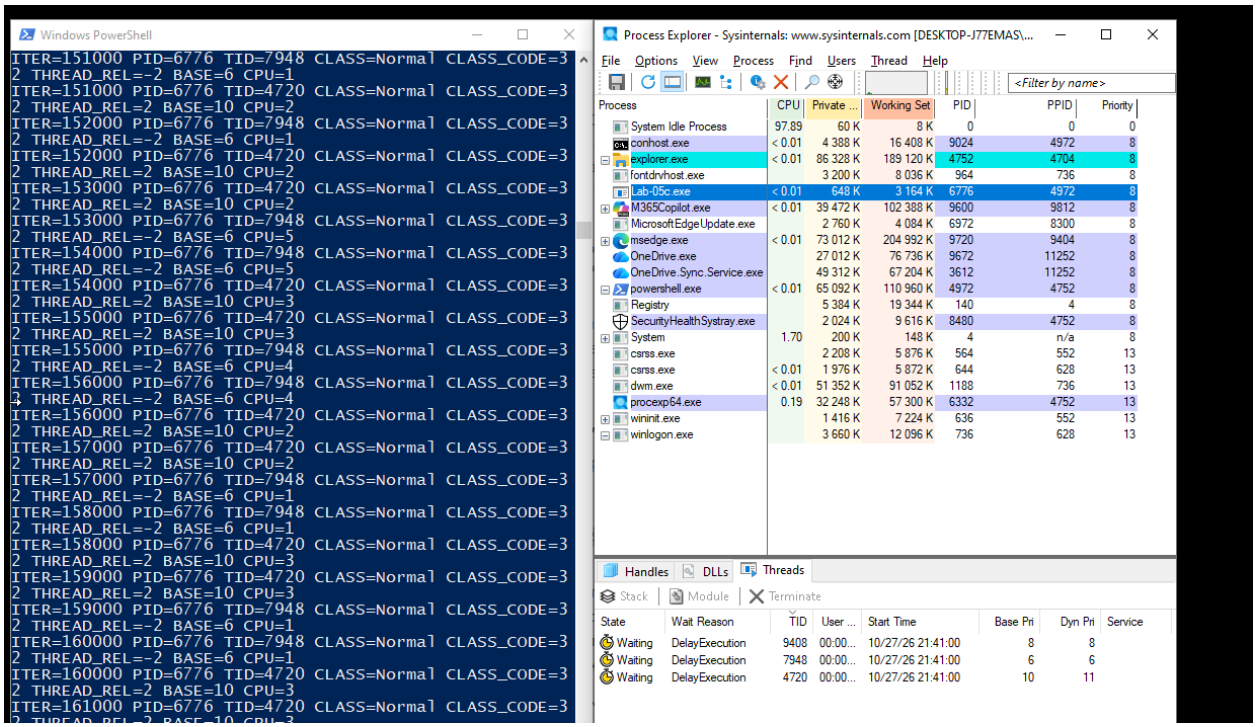


Рисунок 2.11 – Приоритеты и маска Lab-05c, вариант 2

В PID = 6776 поток TID = 7948 имеет THREAD_REL = −2 и BASE = 6, а TID = 4720 — THREAD_REL = 2 и BASE = 10. Эти базовые значения подтверждены Process Explorer. Для TID = 4720 текущий Dyn Pri = 11 отличается от Base Pri = 10; динамическое повышение не меняет заданный относительный уровень. В опыте задан относительный уровень, а наблюдаемое динамическое значение отражает текущее состояние планирования. Их раздельная проверка исключает смешение этих показателей.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 2.12.

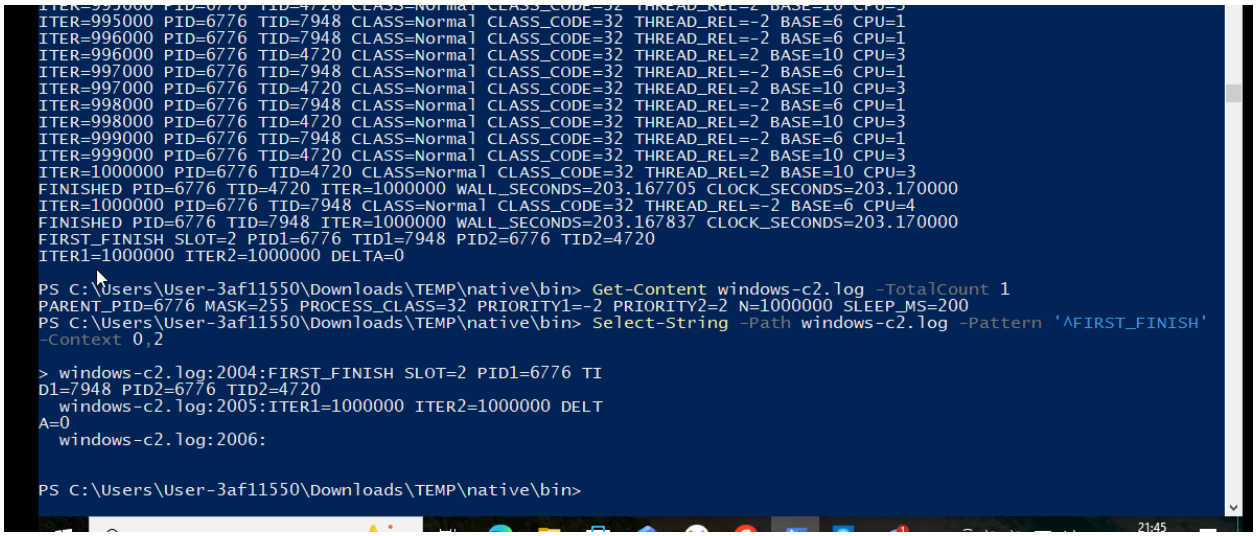


Рисунок 2.12 – Прогресс Lab-05c при первом завершении, вариант 2

Первым зарегистрировано завершение потока Highest: PID = 6776, TID = 4720, SLOT = 2. Получены ITER1 = 1000000, ITER2 = 1000000, DELTA = 0. Его WALL_SECONDS = 203,167705 с, у Lowest — 203,167837 с. При доступности нескольких CPU изменение приоритета не привело к различию зафиксированных счётчиков. Наличие нескольких разрешённых CPU уменьшает необходимость прямой конкуренции готовых потоков за один исполнительный ресурс.

2.4.3 Один CPU и приоритеты Lowest / Highest

Вариант 3: один CPU, приоритеты Lowest / Highest. Параметры командной строки для приоритетов: -2 2. Оба исполнителя имеют один PID и разные TID.

Параметры потоков в Process Explorer представлены на рисунке 2.13.

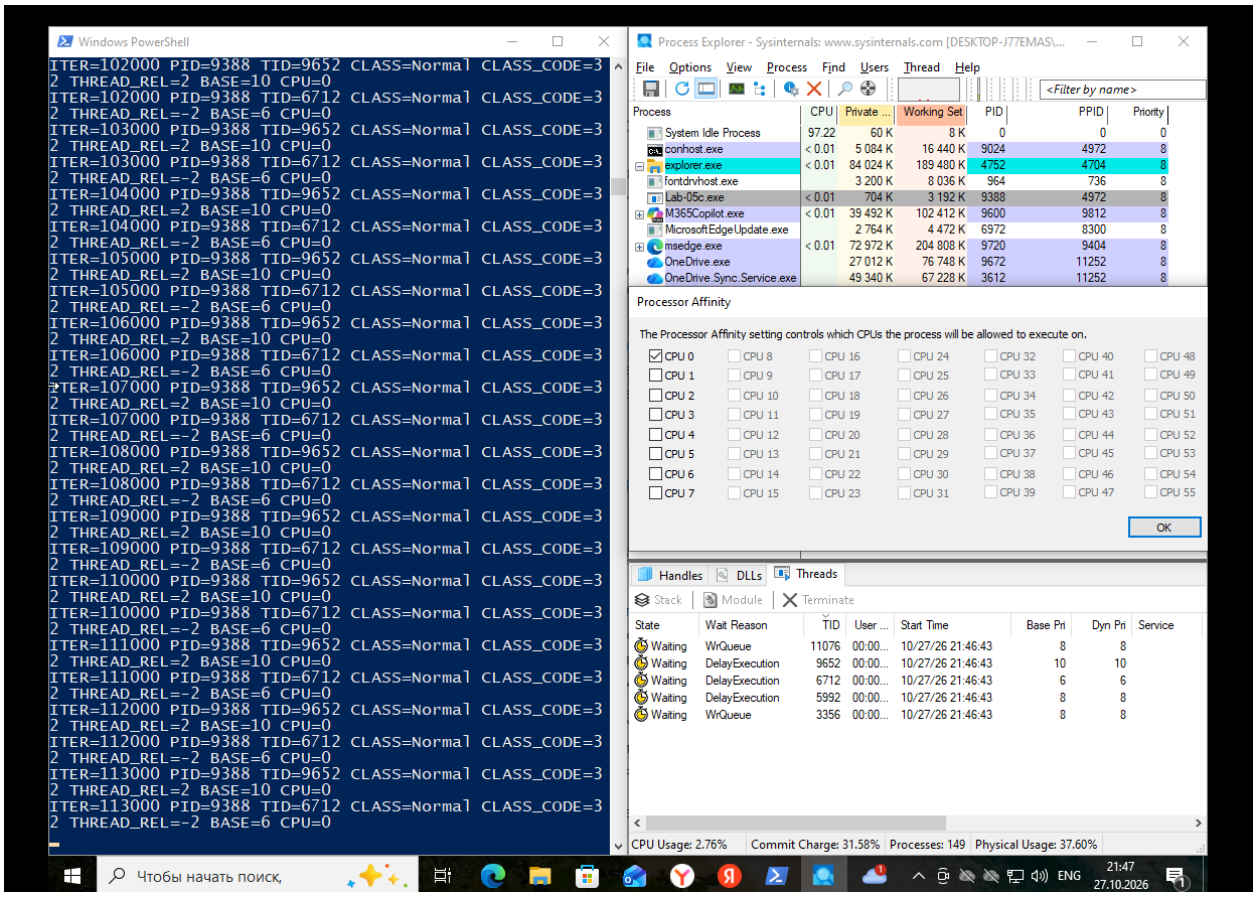


Рисунок 2.13 – Приоритеты и маска Lab-05c, вариант 3

Оба рабочих потока PID = 9388 закреплены за CPU 0 маской 1. TID = 6712 имеет базовый приоритет 6, TID = 9652 — 10. Process Explorer и рабочие строки подтверждают заданные Lowest и Highest и ограничение родственности. Одинаковая маска проверяется для обоих рабочих TID. Это подтверждает, что ограничение относится ко всему сравниваемому набору исполнителей.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 2.14.

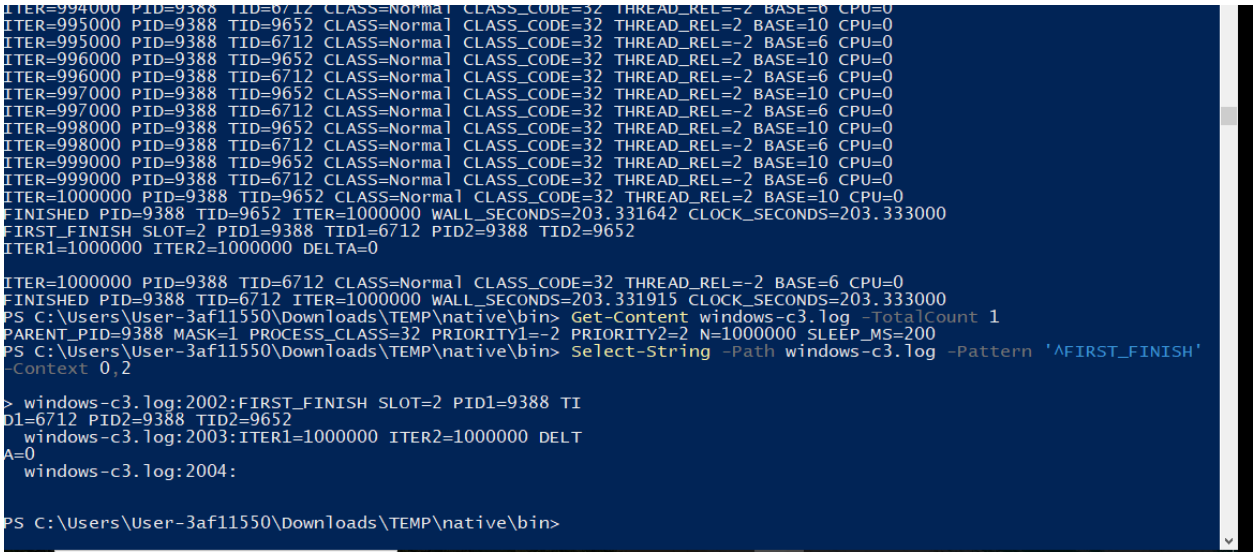


Рисунок 2.14 – Прогресс Lab-05c при первом завершении, вариант 3

Первым завершился Highest: PID = 9388, TID = 9652, SLOT = 2. Счётчики ITER1 и ITER2 равны 1000000, DELTA = 0. Времена Highest и Lowest составили 203,331642 и 203,331915 с. Конкуренция на одном CPU не создала измеримого расхождения итераций: исполнители большую часть времени ожидали таймер. Заданный приоритет влияет на выбор готового потока, но не сокращает установленную программой задержку ожидания.

2.5 Сопоставление результатов Windows

Итоги трёх процессных опытов фиксируются в таблице 2.1, а трёх потоковых — в таблице 2.2. Значения переносятся из FIRST_FINISH, а не из произвольной последней строки консоли.

Таблица 2.1 – Прогресс процессов Windows при первом завершении

Вариант	Первый PID / TID	ITER1	ITER2	DELTA
1	9868 / 1300	999000	1000000	1000
2	5180 / 6924	1000000	1000000	0
3	8088 / 2036	998000	1000000	2000

При одинаковых классах процессов получено расхождение 1000 итераций. Для Below Normal / High оно равно 0 на всех CPU и 2000 на одном CPU. Во всех трёх процессных опытах первым зарегистрирован второй исполнитель. Эти значения характеризуют отдельные запуски и не задают коэффициент ускорения. Порядок вариантов сохраняется при сопоставлении

с отдельными снимками. Разность вычисляется по сохранённым счётчикам, а не по последующему окончательному состоянию процессов.

Результаты потоковых опытов Windows сведены в таблицу 2.2. Во всех вариантах использовался класс процесса Normal, а уровни рабочих потоков задавались отдельно.

Таблица 2.2 – Прогресс потоков Windows при первом завершении

Вариант	**Первый PID / TID**	**ITER1**	**ITER2**	**DELTA**
1	11756 / 5412	1000000	1000000	0
2	6776 / 4720	1000000	1000000	0
3	9388 / 9652	1000000	1000000	0

Во всех потоковых опытах DELTA = 0, первым зарегистрирован второй рабочий поток. Приоритеты подтверждены независимо в Process Explorer. Переход к одному CPU сохранил равенство счётчиков; результат согласуется с преобладанием ожидания таймера над вычислительной работой. Таблица объединяет результаты с различными относительными уровнями потока. Нулевое расхождение не исключает небольшого различия времени выхода из рабочей функции.

В Windows наибольшее наблюдаемое расхождение составило 2000 итераций, то есть 0,2 % заданного объёма работы. Различие процессов и потоков в этой серии не доказывает преимущества одного способа организации: проведено по одному запуску на вариант, а длительность определяется главным образом задержками и условиями гостевой ОС.

3 Исследование планирования в Linux

Linux-версии используют sched_getaffinity, sched_setaffinity, sched_getcpu, getpriority, nice и setpriority. PID возвращает getpid, TID — системный вызов gettid. Для процессного опыта приоритет задаётся nice до exec; для потокового — setpriority в самом рабочем потоке [2, 4].

3.1 Информация о текущем исполнителе

Lab-05a выводит маску процесса, число разрешённых CPU, nice и идентификаторы. Дополнительно выводятся числовая политика планирования и sched_priority. Системная маска, предусмотренная только для Windows, в Linux не выводится.

Контрольный запуск Lab-05a в Linux представлен на рисунке 3.1.

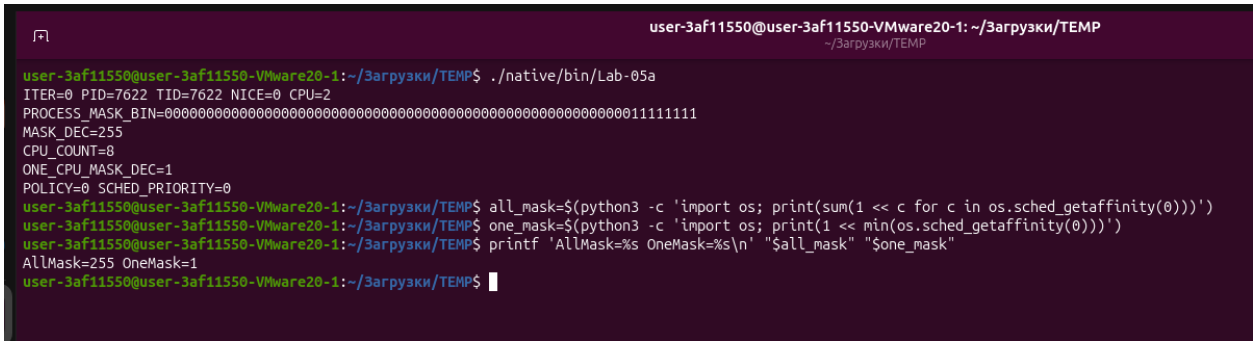


Рисунок 3.1 – Вывод Lab-05a в Linux

В запуске PID = TID = 7622 получены NICE = 0, маска 255, CPU_COUNT = 8 и CPU = 2. Политика POLICY = 0 соответствует SCHED_OTHER, SCHED_PRIORITY = 0. Для этой политики вес обслуживания определяется nice; отдельный приоритет реального времени не задан. Информация о политике и nice описывает разные параметры планирования. Нулевой sched_priority здесь не означает отсутствие учёта веса nice.

3.2 Циклическая рабочая нагрузка

Lab-05x сохраняет тот же цикл и задержку, что и Windows-версия. Одна строка Linux формируется целиком и передаётся одним write, чтобы уменьшить смешивание вывода двух процессов. Сохранение логов позволяет восстановить значения при первом завершении после окончания опыта.

Последние итерации и показатели времени Lab-05x приведены на рисунке 3.2.

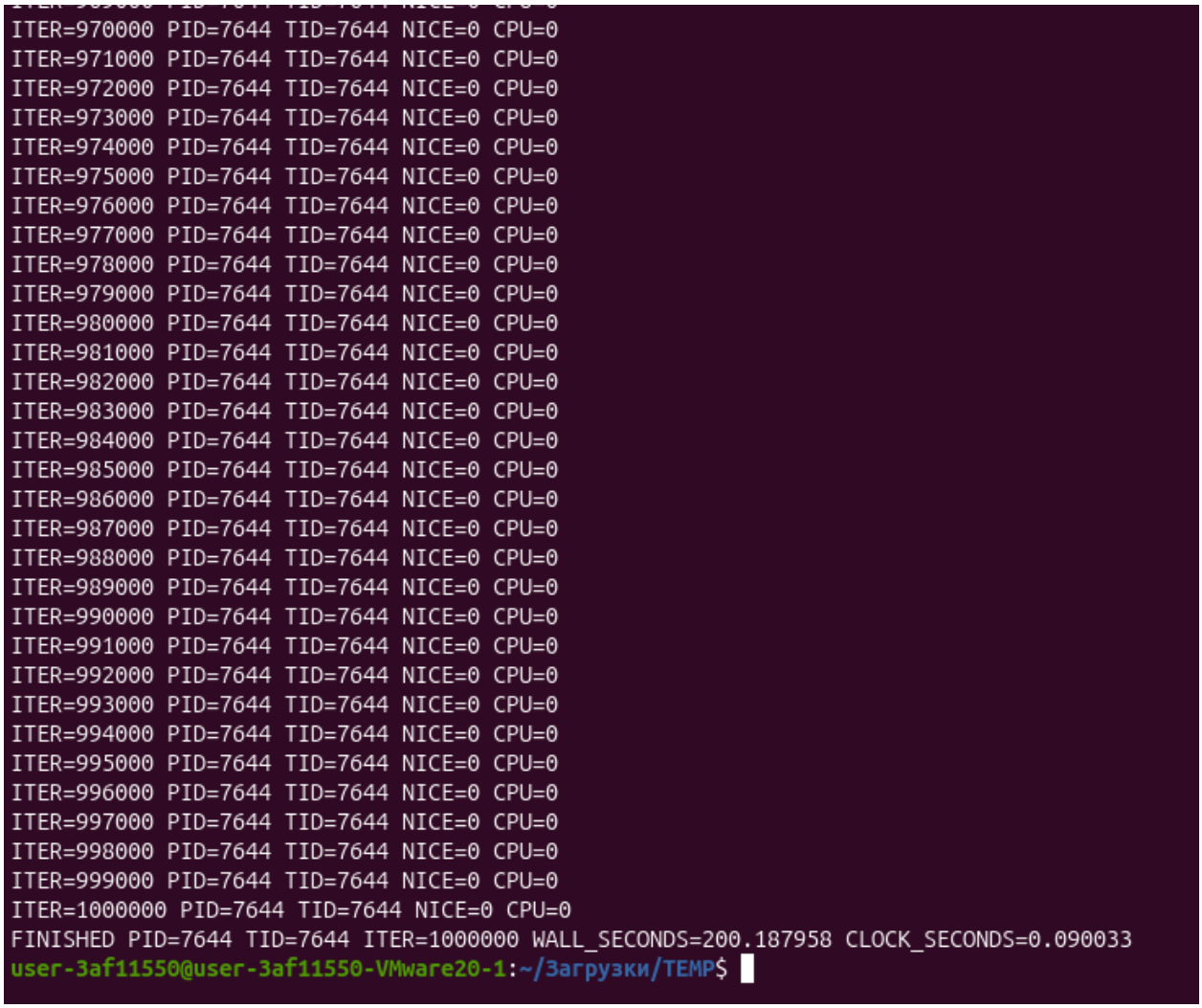


Рисунок 3.2 – Завершение Lab-05x в Linux

Процесс PID = TID = 7644 завершил 1 000 000 итераций: WALL_SECONDS = 200,187958 с, CLOCK_SECONDS = 0,090033 с. В отличие от календарного времени, clock в Linux отражает CPU-время процесса и почти не учитывает ожидание таймера. В показанном фрагменте используется CPU 0; это не доказывает ограничения маски одним CPU. Разность календарного и процессорного времени согласуется с периодическими блокировками. Наблюдение одного номера CPU не заменяет проверки разрешённого набора.

3.3 Два дочерних процесса

Lab-05b запускает два одинаковых дочерних процесса. В каждом варианте используется одинаковая нагрузка, а число итераций сохраняется при первом завершении.

3.3.1 Все доступные CPU и приоритеты 0 / 0

Вариант 1: все доступные CPU, приоритеты 0 / 0. Параметры командной строки для приоритетов: 0 0. Исследуются разные процессы, поэтому необходимо сопоставить оба PID.

Сведения ps и /proc о процессах представлены на рисунке 3.3.

```
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ sudo -v
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ sudo ./native/bin/Lab-05b "$all_mask" 0 0 100
0000 200 > linux-b1.log 2>&1 &
launcher_pid=$!
[2] 7816
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ head -n 5 linux-b1.log
PARENT_PID=7820 MASK=255 P2=0 P3=0 N=1000000 SLEEP_MS=200
CHILD1 PID=7821
CHILD2 PID=7822
ITER=1000 PID=7822 TID=7822 NICE=0 CPU=0
ITER=1000 PID=7821 TID=7821 NICE=0 CPU=5
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ lab_pid=$(sed -n 's/^PARENT_PID=\([0-9]*\).*/
\1/p' linux-b1.log | head -n 1)
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ printf 'Lab PID=%s\n' "$lab_pid"
Lab PID=7820
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ p1=$(sed -n 's/^CHILD1 PID=\([0-9]*\).*/\1/p'
 linux-b1.log | head -n 1)
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ p2=$(sed -n 's/^CHILD2 PID=\([0-9]*\).*/\1/p'
 linux-b1.log | head -n 1)
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ ps -p "$lab_pid,$p1,$p2" -o pid,ppid,cls,rtpr
io,pri,ni,psr,stat,comm
    PID    PPID CLS RTPRIO PRI  NI PSR STAT COMMAND
   7820    7819  TS      -  19   0   3 S    Lab-05b
   7821    7820  TS      -  19   0   5 S    Lab-05x
   7822    7820  TS      -  19   0   0 S    Lab-05x
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ grep -E '^(Name|Pid|Tgid|Cpus_allowed|Cpus_al
lowed_list):' "/proc/$p1/status" "/proc/$p2/status"
/proc/7821/status:Name: Lab-05x
/proc/7821/status:Tgid: 7821
/proc/7821/status:Pid:  7821
/proc/7821/status:Cpus_allowed: ff
/proc/7821/status:Cpus_allowed_list:    0-7
/proc/7822/status:Name: Lab-05x
/proc/7822/status:Tgid: 7822
/proc/7822/status:Pid:  7822
/proc/7822/status:Cpus_allowed: ff
/proc/7822/status:Cpus_allowed_list:    0-7
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ awk '{print FILENAME, "priority=" $18, "nice=
" $19}' "/proc/$p1/stat" "/proc/$p2/stat"
/proc/7821/stat priority=20 nice=0
/proc/7822/stat priority=20 nice=0
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$
```

Рисунок 3.3 – Приоритеты и маска Lab-05b, вариант 1

Родитель имеет PID = 7820, дочерние процессы — 7821 и 7822. У обоих NI = 0, Cpus_allowed = ff и Cpus_allowed_list = 0–7. В /proc/PID/stat показаны priority = 20 и nice = 0, тогда как ps выводит PRI = 19; эти представления приоритета не следует смешивать. Проверка отдельных PID подтверждает настройки каждого дочернего процесса. Совпадение разрешённых наборов обеспечивает одинаковые ограничения родственности.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 3.4.

```
/proc/7822/stat priority=20 nice=0
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ while ! grep -q '^FIRST_FINISH' linux-b1.log;
 do if grep -q '^ERROR:' linux-b1.log; then tail -n 10 linux-b1.log; break; fi; sleep 1; done
[1]-  Завершён           sudo ./native/bin/Lab-05b "$all_mask" 0 0 1000000 200 > linux-b1.log 2>&1
[2]+  Завершён           sudo ./native/bin/Lab-05b "$all_mask" 0 0 1000000 200 > linux-b1.log 2>&1
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ head -n 1 linux-b1.log
PARENT_PID=7820 MASK=255 P2=0 P3=0 N=1000000 SLEEP_MS=200
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ grep -A 1 '^FIRST_FINISH' linux-b1.log
FIRST_FINISH SLOT=2 PID1=7821 TID1=7821 PID2=7822 TID2=7822
ITER1=1000000 ITER2=1000000 DELTA=0
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$
```

Рисунок 3.4 – Прогресс Lab-05b при первом завершении, вариант 1

Первым зарегистрирован второй процесс: PID = TID = 7822, SLOT = 2. Сохранены ITER1 = 1000000, ITER2 = 1000000, DELTA = 0. При одинаковом nice и восьми разрешённых CPU оба процесса достигли одинакового прогресса к моменту первого завершения. Нулевой DELTA относится к сохранённому снимку счётчиков. Он не утверждает полного совпадения моментов начала и завершения обоих процессов.

3.3.2 Все доступные CPU и приоритеты 19 / −20

Вариант 2: все доступные CPU, приоритеты 19 / −20. Параметры командной строки для приоритетов: 19 -20. Исследуются разные процессы, поэтому необходимо сопоставить оба PID.

Сведения ps и /proc о процессах представлены на рисунке 3.5.

```
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ sudo -v
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ sudo ./native/bin/Lab-05b "$all_mask" 19 -20 1000000 200 > linux-b2.log 2>&1 &
launcher_pid=$!
[2] 8062
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ head -n 5 linux-b2.log
PARENT_PID=8066 MASK=255 P2=19 P3=-20 N=1000000 SLEEP_MS=200
CHILD1 PID=8067
CHILD2 PID=8068
ITER=1000 PID=8067 TID=8067 NICE=19 CPU=6
ITER=1000 PID=8068 TID=8068 NICE=-20 CPU=4
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ lab_pid=$(sed -n 's/^PARENT_PID=\([0-9]*\).*/\1/p' linux-b2.log | head -n 1)
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ printf 'Lab PID=%s\n' "$lab_pid"
Lab PID=8066
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ p1=$(sed -n 's/^CHILD1 PID=\([0-9]*\).*/\1/p' linux-b2.log | head -n 1)
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ p2=$(sed -n 's/^CHILD2 PID=\([0-9]*\).*/\1/p' linux-b2.log | head -n 1)
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ ps -p "$lab_pid,$p1,$p2" -o pid,ppid,cls,rtprio,pri,ni,psr,stat,comm
    PID    PPID CLS RTPRIO PRI  NI PSR STAT COMMAND
   8066    8065  TS      -  19   0   2 S    Lab-05b
   8067    8066  TS      -   0  19   6 SN   Lab-05x
   8068    8066  TS      -  39 -20   4 S<   Lab-05x
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ grep -E '^(Name|Pid|Tgid|Cpus_allowed|Cpus_allowed_list):' "/proc/$p1/status" "/proc/$p2/status"
/proc/8067/status:Name: Lab-05x
/proc/8067/status:Tgid: 8067
/proc/8067/status:Pid:  8067
/proc/8067/status:Cpus_allowed: ff
/proc/8067/status:Cpus_allowed_list:    0-7
/proc/8068/status:Name: Lab-05x
/proc/8068/status:Tgid: 8068
/proc/8068/status:Pid:  8068
/proc/8068/status:Cpus_allowed: ff
/proc/8068/status:Cpus_allowed_list:    0-7
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ awk '{print FILENAME, "priority=" $18, "nice=" $19}' "/proc/$p1/stat" "/proc/$p2/stat"
/proc/8067/stat priority=39 nice=19
/proc/8068/stat priority=0 nice=-20
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$
```

Рисунок 3.5 – Приоритеты и маска Lab-05b, вариант 2

Родительский PID = 8066, дочерние PID = 8067 и 8068. Для них подтверждены NI = 19 и −20, в /proc — nice = 19 и −20, priority = 39 и 0

соответственно. Обе маски равны ff и разрешают CPU 0–7; отрицательное nice действительно установлено. Корректная установка отрицательного nice подтверждается состоянием процесса после запуска. Само значение в командной строке такой проверки не заменяет.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 3.6.

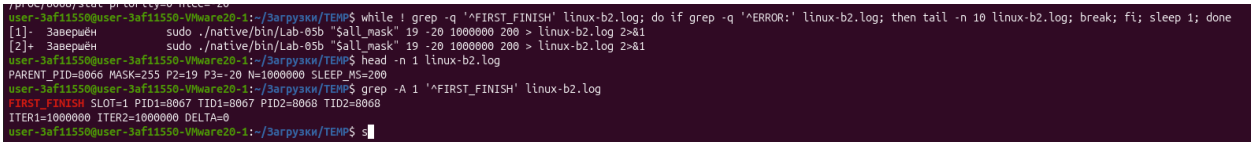


Рисунок 3.6 – Прогресс Lab-05b при первом завершении, вариант 2

Первым зарегистрирован процесс с nice = 19: PID = TID = 8067, SLOT = 1. При этом ITER1 = 1000000, ITER2 = 1000000, DELTA = 0. Такой порядок допустим: высокий вес CPU не отменяет блокировку на таймере и не гарантирует более раннего завершения при наличии свободных CPU. При периодическом ожидании порядок пробуждения и последнего выхода может отличаться от порядка весов обслуживания готовых задач.

3.3.3 Один CPU и приоритеты 19 / −20

Вариант 3: один CPU, приоритеты 19 / −20. Параметры командной строки для приоритетов: 19 -20. Исследуются разные процессы, поэтому необходимо сопоставить оба PID.

Сведения ps и /proc о процессах представлены на рисунке 3.7.

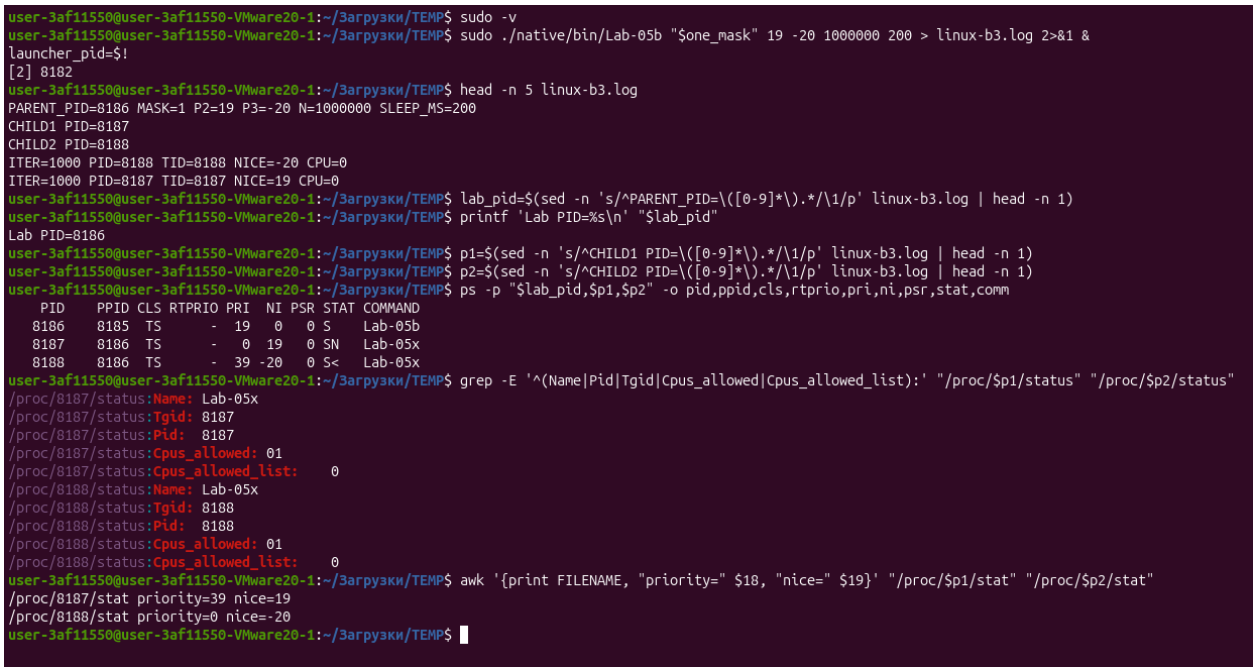


Рисунок 3.7 – Приоритеты и маска Lab-05b, вариант 3

Родительский PID = 8186, дочерние PID = 8187 и 8188. Значения NI = 19 и −20 совпадают с параметрами запуска. В обеих записях /proc показаны

Cpus_allowed = 01 и Cpus_allowed_list = 0; рабочие строки также содержат CPU = 0. Согласованность маски в /proc и номера CPU в рабочем выводе подтверждает фактическое выполнение опыта с одним разрешённым CPU.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 3.8.

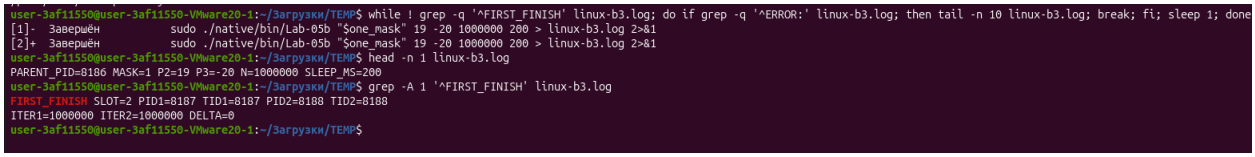


Рисунок 3.8 – Прогресс Lab-05b при первом завершении, вариант 3

Первым зарегистрирован процесс с nice = −20: PID = TID = 8188, SLOT = 2. Оба счётчика равны 1000000, DELTA = 0. Ограничение одним CPU не создало отставания по счётчику: периоды ожидания освобождали процессор для другого исполнителя. Пока один исполнитель ожидает задержку, другой может продолжать работу на том же CPU. Поэтому ограничение маски само по себе не гарантирует расхождения прогресса.

3.4 Два дочерних потока

Lab-05c запускает два одинаковых рабочих потока. В каждом варианте используется одинаковая нагрузка, а число итераций сохраняется при первом завершении.

3.4.1 Все доступные CPU и приоритеты 0 / 0

Вариант 1: все доступные CPU, приоритеты 0 / 0. Параметры командной строки для приоритетов: 0 0. Оба исполнителя имеют один PID и разные TID.

Сведения ps и /proc о потоках представлены на рисунке 3.9.

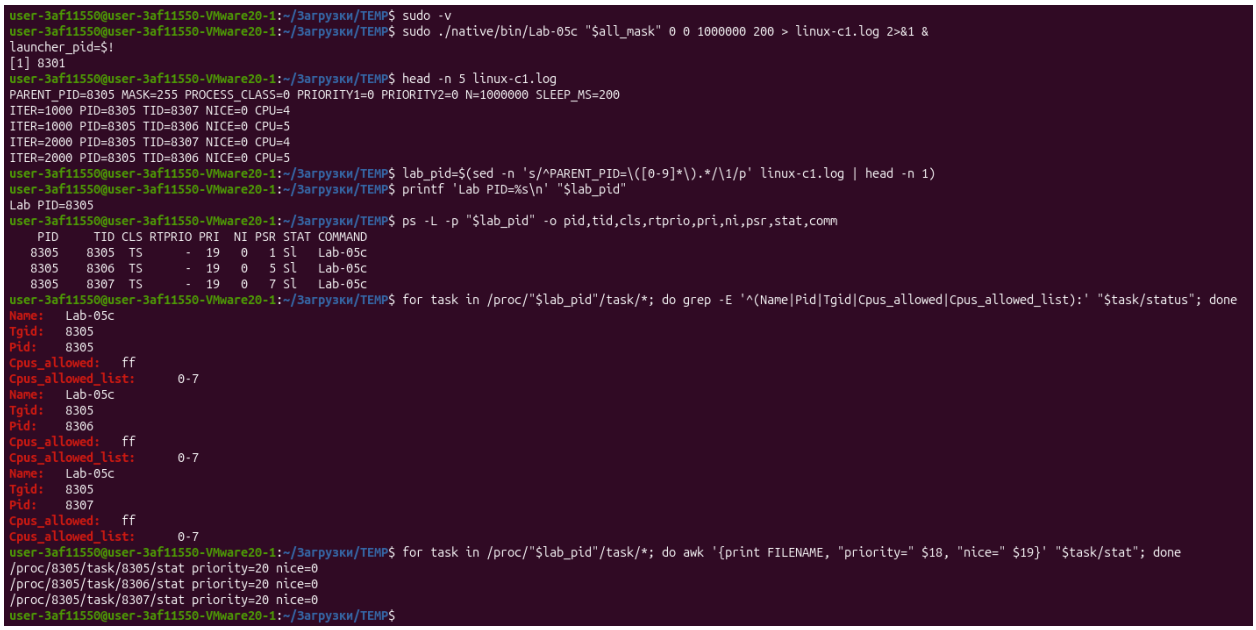


Рисунок 3.9 – Приоритеты и маска Lab-05c, вариант 1

Родительский PID = 8305; рабочие потоки имеют TID = 8306 и 8307. Команда ps -L показывает NI = 0 для обоих, отдельные записи /proc/8305/task/TID подтверждают маску ff и CPU 0–7. Общий Tgid = 8305 отличает потоки одного процесса от отдельных процессов. Для потоковой версии единицей проверки приоритета служит TID. Общий идентификатор группы не позволяет восстановить параметры каждого рабочего потока.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 3.10.

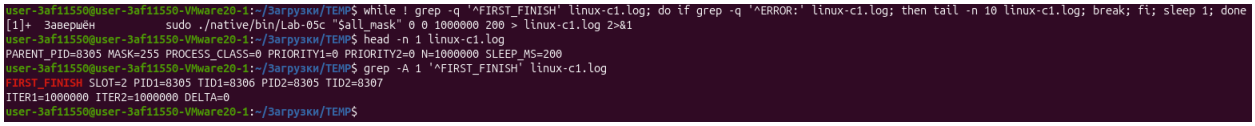


Рисунок 3.10 – Прогресс Lab-05c при первом завершении, вариант 1

Первым зарегистрирован второй поток: PID = 8305, TID = 8307, SLOT = 2. Значения ITER1 = 1000000 и ITER2 = 1000000 совпали, DELTA = 0. При одинаковом nice существенного различия прогресса в данном запуске не выявлено. Совпадение прогресса показывает одинаковый объём выполненных циклов на момент фиксации, а не идентичное расписание переключений.

3.4.2 Все доступные CPU и приоритеты 19 / −20

Вариант 2: все доступные CPU, приоритеты 19 / −20. Параметры командной строки для приоритетов: 19 -20. Оба исполнителя имеют один PID и разные TID.

Сведения ps и /proc о потоках представлены на рисунке 3.11.

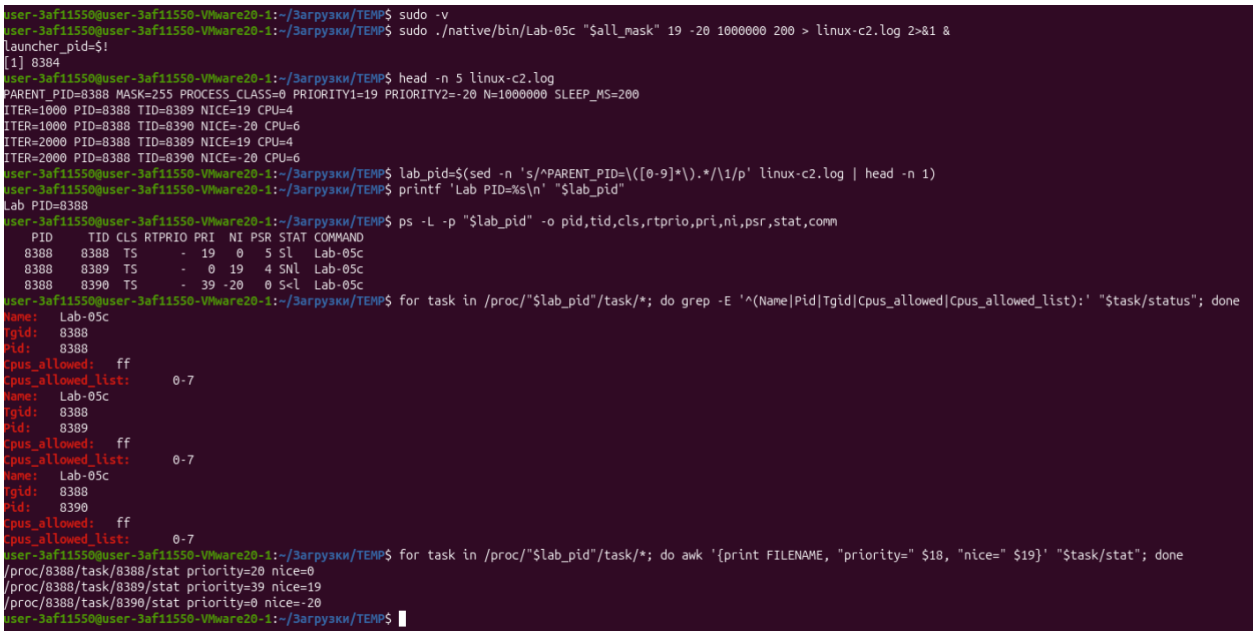


Рисунок 3.11 – Приоритеты и маска Lab-05c, вариант 2

Рабочие потоки PID = 8388 имеют TID = 8389 и 8390. В ps -L и /proc подтверждены nice = 19 и −20, маска ff и разрешённые CPU 0–7. Главный поток сохраняет nice = 0, поэтому проверка только строки процесса не заменяет проверку отдельных TID. Сохранение исходного nice главного потока показывает, что изменение выполнено адресно внутри рабочих потоков.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 3.12.

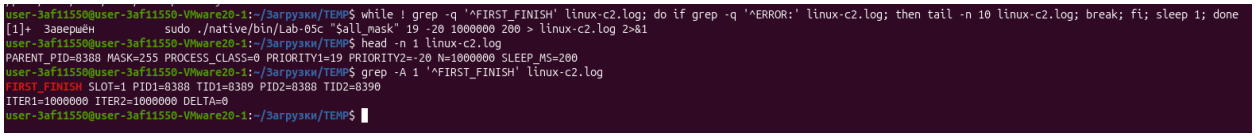


Рисунок 3.12 – Прогресс Lab-05c при первом завершении, вариант 2

Первым зарегистрирован поток с nice = 19: PID = 8388, TID = 8389, SLOT = 1. Сохранены ITER1 = 1000000, ITER2 = 1000000 и DELTA = 0. Различные веса обслуживания не привели к различию счётчиков при нагрузке с регулярными задержками. Назначение разного nice подтверждено независимо от результата измерения. Отсутствие расхождения не означает, что параметры запуска были проигнорированы.

3.4.3 Один CPU и приоритеты 19 / −20

Вариант 3: один CPU, приоритеты 19 / −20. Параметры командной строки для приоритетов: 19 -20. Оба исполнителя имеют один PID и разные TID.

Сведения ps и /proc о потоках представлены на рисунке 3.13.

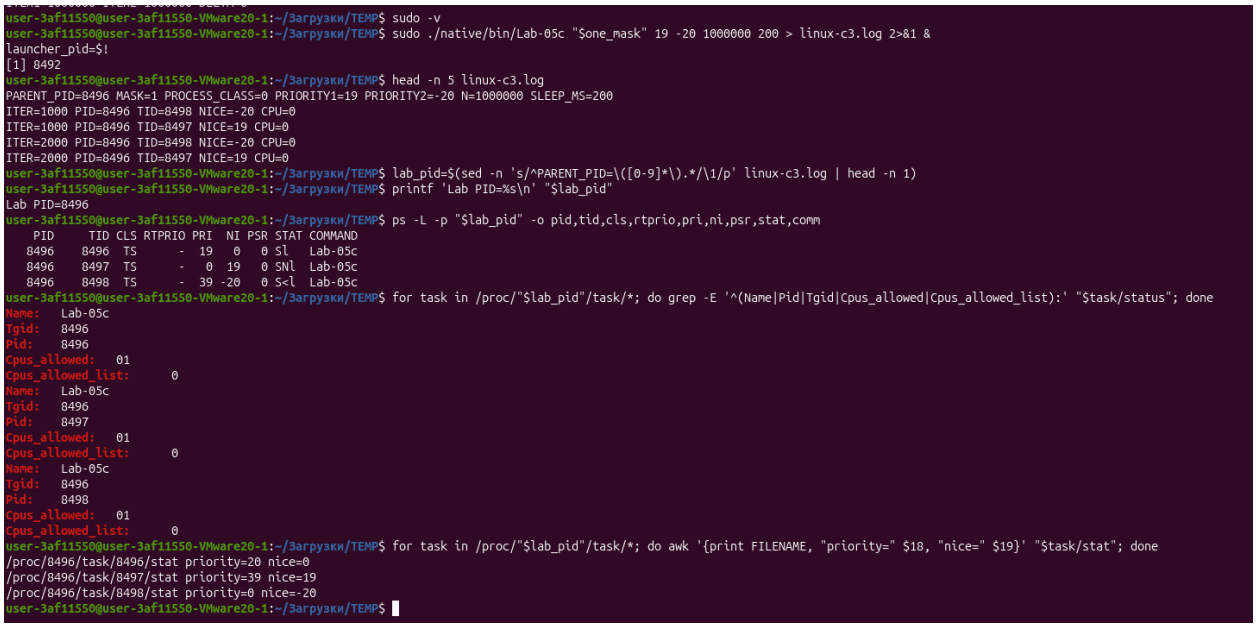


Рисунок 3.13 – Приоритеты и маска Lab-05c, вариант 3

Для PID = 8496 рабочие TID = 8497 и 8498 имеют nice = 19 и −20. Маска 01 и Cpus_allowed_list = 0 подтверждены отдельно для главного и рабочих потоков. В рабочих строках обоих исполнителей показан CPU = 0. Проверка главного и дочерних TID исключает ситуацию, когда ограничение применено только к одному из участников опыта.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 3.14.

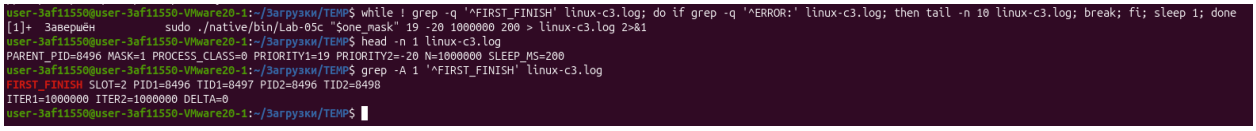


Рисунок 3.14 – Прогресс Lab-05c при первом завершении, вариант 3

Первым зарегистрирован поток с nice = −20: PID = 8496, TID = 8498, SLOT = 2. Оба счётчика равны 1000000, DELTA = 0. Совместное использование одного CPU сохранило одинаковый прогресс по принятой метрике. Полученный результат согласуется с характером нагрузки, в которой короткие вычислительные участки разделяются длительным ожиданием таймера.

3.5 Сопоставление процессов и потоков Linux

Итоги процессных запусков представлены в таблице 3.1, потоковых — в таблице 3.2. Приоритеты проверяются во время работы; для потоков считываются отдельные записи /proc/PID/task/TID.

Таблица 3.1 – Прогресс процессов Linux при первом завершении

Вариант	Первый PID / TID	ITER1	ITER2	DELTA
1	7822 / 7822	1000000	1000000	0
2	8067 / 8067	1000000	1000000	0
3	8188 / 8188	1000000	1000000	0

Во всех трёх процессных опытах DELTA = 0. При nice 0 / 0 и при маске одного CPU первым зарегистрирован второй процесс; при nice 19 / −20 на всех CPU — первый. Порядок завершения сам по себе не определяет преимущество приоритета при равных счётчиках. Сравнение включает как одинаковые, так и различные nice. Во всех случаях исходные настройки проверялись по состоянию работающих процессов.

Итоги трёх потоковых опытов Linux приведены в таблице 3.2. Первый исполнитель определяется по полю SLOT с учётом соответствующего TID.

Таблица 3.2 – Прогресс потоков Linux при первом завершении

Вариант	Первый PID / TID	ITER1	ITER2	DELTA
1	8305 / 8307	1000000	1000000	0
2	8388 / 8389	1000000	1000000	0
3	8496 / 8498	1000000	1000000	0

Во всех трёх потоковых опытах DELTA = 0. Последовательность зарегистрированных первых исполнителей совпала с процессной серией: второй, первый, второй. Проверки отдельных TID подтвердили фактические nice и маски. Итоговые значения относятся к двум рабочим потокам. Параметры главного потока не подменяют значения, установленные для дочерних TID.

В Linux и процессы, и потоки достигли одинаковых счётчиков при первом завершении во всех вариантах. В Windows у процессов получены DELTA = 1000, 0 и 2000, у потоков — только 0. Эти единичные измерения не позволяют ранжировать ОС по скорости; определения CLOCK_SECONDS различаются, а WALL_SECONDS включает периодическое ожидание.

4 Моделирование алгоритмов планирования

Модели SJF, RR и CP обрабатывают один и тот же набор из десяти задач. Используются целочисленные единицы модельного времени; их физическая длительность не задаётся. Одновременно обслуживается одна задача. Стоимость переключения принята равной нулю, ввод-вывод в данном наборе отсутствует. Исходные данные представлены в таблице 4.1.

Таблица 4.1 – Исходные задачи

Задача	Поступление	Выполнение	Приоритет
1	0	80	3
2	20	45	5
3	30	60	2
4	50	30	4
5	70	90	1
6	100	55	3
7	120	75	2
8	150	40	5
9	180	65	4
10	200	85	1

Большему числу приоритета соответствует более высокий приоритет. Задачи с одинаковым временем поступления упорядочиваются по ID. Новые поступления на границе кванта включаются в очередь до возврата задачи с истёкшим квантом. Эти соглашения исключают неоднозначность воспроизведения. Набор поступлений и CPU-потребностей одинаков для всех трёх алгоритмов. Поэтому сравнение отражает изменение порядка обслуживания при неизменной исходной работе.

4.1 Определение статистики

Для каждой задачи известны время поступления a, необходимое CPU-время b, момент первого запуска s и момент окончания c. Оборотное время равно $c - a$; время ожидания равно $c - a - b$; время отклика равно $s - a$. Средние значения рассчитываются по всем десяти задачам. Пропускная способность равна числу завершённых задач, делённому на время от первого поступления до последнего завершения [7].

В настоящей работе «время выполнения» означает предоставленное задаче процессорное время b. Оно не смешивается с оборотным временем. Суммарная CPU-потребность равна 625, среднее выполнение — 62,5. При нулевой стоимости переключений и отсутствии простоя полное время равно 625, пропускная способность — 0,016 задачи на единицу времени для всех трёх алгоритмов.

SJF выбирает кратчайшую из уже поступивших задач и не вытесняет текущую. RR обслуживает готовые задачи циклически с заданным квантом. CP немедленно вытесняет текущую задачу при поступлении более приоритетной; внутри одного уровня действует RR. Прерванная задача возвращается в начало очереди своего уровня и при следующем запуске получает новый квант.

4.2 Алгоритм Shortest Job First

Для заданного набора невытесняющий SJF строит порядок 1 → 4 → 2 → 8 → 6 → 3 → 9 → 7 → 10 → 5. Короткая новая задача не прерывает уже выполняющуюся. Первый процесс занимает интервал [0; 80), последний заканчивается в момент 625.

Построенный план и индивидуальные времена задач представлены на рисунке 4.1.

```
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ python3 simulation/Lab-05SJF.py
ALGORITHM=SJF QUANTUM=0 SWITCH_COST=0 HIGHER_NUMBER_HIGHER_PRIORITY
ID ARR CPU PRI START FINISH WAIT TURN RESPONSE
    1    0   80   3     0    80    0   80    0
    2   20   45   5   110   155   90  135   90
    3   30   60   2   250   310  220  280  220
    4   50   30   4    80   110   30   60   30
    5   70   90   1   535   625  465  555  465
    6  100   55   3   195   250   95  150   95
    7  120   75   2   375   450  255  330  255
    8  150   40   5   155   195    5   45    5
    9  180   65   4   310   375  130  195  130
   10  200   85   1   450   535  250  335  250
algorithm=SJF quantum=0 avg_waiting=154 avg_execution=62.500000 avg_turnaround=216.500000 avg_response=154 max_response=465 throughput=0.016000 makespan=625 switches=9
GANTT: [0,80):P1 [80,110):P4 [110,155):P2 [155,195):P8 [195,250):P6 [250,310):P3 [310,375):P9 [375,450):P7 [450,535):P10 [535,625):P5
Results: /home/user-3af11550/Загрузки/TEMP/simulation/results
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$
```

Рисунок 4.1 – Расписание и статистика SJF

Среднее ожидание для данного набора равно 154,0, среднее оборотное время — 216,5. Результат относится к указанным поступлениям и невытесняющему варианту SJF; он не означает универсальной оптимальности для произвольных поступлений. Выбор следующей задачи выполняется только среди уже поступивших. Более короткая задача, появившаяся позднее, не прерывает начавшееся обслуживание.

4.3 Алгоритм Round Robin

Для RR рассматриваются кванты 5, 10, 20, 40 и 80. Меньший квант уменьшает ожидание первого обслуживания, но увеличивает число переключений. В модели цена переключения равна нулю, поэтому дополнительные переключения сами по себе не увеличивают полное время обработки.

Расписание RR при кванте 80 приведено на рисунке 4.2.

```
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ python3 simulation/Lab-05RR.py --quantum 80
ALGORITHM=RR QUANTUM=80 SWITCH_COST=0 HIGHER_NUMBER_HIGHER_PRIORITY
ID ARR CPU PRI START FINISH WAIT TURN RESPONSE
    1    0   80   3     0    80    0   80     0
    2   20   45   5    80   125   60  105    60
    3   30   60   2   125   185   95  155    95
    4   50   30   4   185   215  135  165   135
    5   70   90   1   215   620  460  550   145
    6  100   55   3   295   350  195  250   195
    7  120   75   2   350   425  230  305   230
    8  150   40   5   425   465  275  315   275
    9  180   65   4   465   530  285  350   285
   10  200   85   1   530   625  340  425   330
algorithm=RR quantum=80 avg_waiting=207.500000 avg_execution=62.500000 avg_turnaround=270 avg_response=175 max_response=330 throughput=0.016000 makespan=625 switches=11
GANTT: [0,80):P1 [80,125):P2 [125,185):P3 [185,215):P4 [215,295):P5 [295,350):P6 [350,425):P7 [425,465):P8 [465,530):P9 [530,610):P10 [610,620):P5 [620,625):P10
Results: /home/user-3af11550/Загрузки/TEMP/simulation/results
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$
```

Рисунок 4.2 – Расписание Round Robin при выбранном кванте

Квант 80 выбран по минимуму среднего ожидания среди исследованных значений. При нём среднее ожидание равно 207,5, оборотное время — 270,0; средний отклик 175,0 заметно хуже, чем при кванте 5. Увеличение кванта уменьшает число прерываний длинных задач, однако новые короткие задачи могут дольше ждать первого предоставления CPU.

4.4 Вытесняющее приоритетное планирование

В CP приоритет учитывается прежде квантования: задача более высокого уровня вытесняет текущую немедленно. При равных приоритетах применяется циклическое обслуживание. Значения 5, 10, 20, 40 и 80 проверяются на одном наборе задач и с одинаковыми соглашениями.

Расписание CP при кванте 80 представлено на рисунке 4.3.

```
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ python3 simulation/Lab-05CP.py --quantum 80
ALGORITHM=CP QUANTUM=80 SWITCH_COST=0 HIGHER_NUMBER_HIGHER_PRIORITY
ID ARR CPU PRI START FINISH WAIT TURN RESPONSE
    1    0   80   3     0   260   180   260     0
    2   20   45   5    20    65     0    45     0
    3   30   60   2   315   375   285   345   285
    4   50   30   4    65    95    15    45    15
    5   70   90   1   450   620   460   550   380
    6  100   55   3   260   315   160   215   160
    7  120   75   2   375   450   255   330   255
    8  150   40   5   150   190     0    40     0
    9  180   65   4   190   255    10    75    10
   10  200   85   1   530   625   340   425   330
algorithm=CP quantum=80 avg_waiting=170.500000 avg_execution=62.500000 avg_turnaround=233 avg_response=143.500000 max_response=380 throughput=0.016000 makespan=625 switches=13
GANTT: [0,20):P1 [20,65):P2 [65,95):P4 [95,150):P1 [150,190):P8 [190,255):P9 [255,260):P1 [260,315):P6 [315,375):P3 [375,450):P7 [450,530):P5 [530,610):P10 [610,620):P5 [620,625):P10
Results: /home/user-3af11550/Загрузки/TEMP/simulation/results
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$
```

Рисунок 4.3 – Расписание приоритетного алгоритма

При выбранном кванте среднее ожидание равно 170,5, оборотное время — 233,0. Повышение приоритета сокращает задержку более приоритетных задач, но не гарантирует хорошего отклика низкоприоритетным задачам. Появление задачи более высокого приоритета проверяется в каждом такте модели. Равноприоритетные готовые задачи обслуживаются циклически.

4.5 Выбор кванта и сравнение алгоритмов

Критерий выбора кванта — минимальное среднее ожидание; при равенстве выбирается меньший средний отклик, затем меньшее число переключений. Этот критерий ориентирован на данный пакет задач.

Оптимум ищется только среди пяти проверенных значений, а не среди всех возможных квантов. Результаты перебора приведены в таблице 4.2.

Таблица 4.2 – Влияние кванта на модельные показатели

Алгоритм	**Квант**	**Среднее ожидание**	**Средний оборот**	**Средний отклик**	**Переклю-чения**
RR	5	307,5	370,0	18,0	118
RR	10	297,5	360,0	35,0	61
RR	20	279,5	342,0	67,0	33
RR	40	262,0	324,5	118,5	19
RR	80	207,5	270,0	175,0	11
CP	5	181,0	243,5	114,5	86
CP	10	180,0	242,5	116,0	46
CP	20	178,5	241,0	119,0	27
CP	40	178,0	240,5	125,0	18
CP	80	170,5	233,0	143,5	13

Для RR и CP минимальное среднее ожидание в исследованном наборе достигается при кванте 80. Для интерактивной системы такой выбор не является безусловным: квант 5 даёт лучший отклик, но большее ожидание завершения. Выбранный квант определяется заявленным критерием. Минимум среднего ожидания не совпадает с минимумом задержки первого ответа.

4.6 Итоговое сопоставление

В итоговое сопоставление включены SJF и варианты RR и CP при выбранном кванте 80. Обязательные статистические показатели представлены в таблице 4.3.

Таблица 4.3 – Сравнение SJF RR и CP

Алгоритм	Среднее ожидание	Среднее выполнение	Средний оборот	Пропускная способность
SJF	154	62,5	216,5	0,016
RR	207,5	62,5	270	0,016
CP	170,5	62,5	233	0,016

SJF имеет наименьшее среднее ожидание среди трёх сравниваемых расписаний. CP учитывает важность задач и занимает промежуточное положение по среднему ожиданию. RR обеспечивает регулярный доступ к CPU; его преимущества по отклику особенно заметны при небольшом кванте. Равенство среднего времени выполнения связано с одинаковыми CPU-потребностями задач. Алгоритм меняет ожидание, но не исходную длительность полезного обслуживания.

Вывод программы сравнения приведён на рисунке 4.4.

```
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$ python3 simulation/scheduler.py compare
ALG   Q   AVG_WAIT AVG_EXEC AVG_TURN AVG_RESP MAX_RESP SWITCHES THROUGHPUT
SJF   0    154.00    62.50   216.50   154.00      465        9 0.016000
RR    5    307.50    62.50   370.00    18.00       40      118 0.016000
RR   10    297.50    62.50   360.00    35.00       75       61 0.016000
RR   20    279.50    62.50   342.00    67.00      135       33 0.016000
RR   40    262.00    62.50   324.50   118.50      215       19 0.016000
RR   80    207.50    62.50   270.00   175.00      330       11 0.016000
CP    5    181.00    62.50   243.50   114.50      380       86 0.016000
CP   10    180.00    62.50   242.50   116.00      380       46 0.016000
CP   20    178.50    62.50   241.00   119.00      380       27 0.016000
CP   40    178.00    62.50   240.50   125.00      380       18 0.016000
CP   80    170.50    62.50   233.00   143.50      380       13 0.016000
Selection rule: minimum average waiting, then minimum average response, then fewer switches.
BEST_RR_QUANTUM=80
BEST_CP_QUANTUM=80
Results: /home/user-3af11550/Загрузки/TEMP/simulation/results
user-3af11550@user-3af11550-VMware20-1:~/Загрузки/TEMP$
```

Рисунок 4.4 – Сводная проверка квантов и статистики

Выбор кванта 80 для RR и CP согласуется с таблицами 4.2 и 4.3. Среднее CPU-время равно 62,5, пропускная способность — 0,016 для всех вариантов. При нулевой цене переключений одинаковый объём полезной работы сохраняет длительность обслуживания набора.

5 Многоуровневая очередь с обратной связью

Lab-05MQ моделирует 100 процессов с различными поступлениями, потребностью в CPU и характером нагрузки. Все новые процессы попадают в Q0. Исчерпание кванта Q0 переводит процесс в Q1, исчерпание Q1 — в Q2. После полного кванта Q2 равновероятно выбирается Q0, Q1 либо Q2; выбор Q2 не считается перемещением между очередями.

5.1 Параметры и правила модели

Генератор воспроизводимой псевдослучайной последовательности MT19937 инициализируется seed = 525. Время поступления выбирается из [0; 200] мс, CPU-потребность CPU-bound — из [120; 600] мс, I/O-bound — из [80; 250] мс. Для I/O-bound два из трёх вариантов CPU-фазы имеют длину 2–8 мс, один — 24–35 мс; задержка I/O составляет 10–40 мс. Блокировка не расходует CPU, после I/O процесс повышается на один уровень, не выше Q0.

Параметры очередей представлены в таблице 5.1. Распределение между очередями реализовано календарём из десяти слотов по 20 мс: пять слотов Q0, три Q1 и два Q2. Получается период 200 мс с бюджетами 100, 60 и 40 мс. Неиспользованный бюджет пустой очереди передаётся первой непустой очереди начиная с Q0. Поэтому итоговые доли CPU за весь опыт могут отличаться от 50/30/20.

Таблица 5.1 – Параметры многоуровневых очередей

Очередь	Квант мс	Алгоритм	Плановая доля CPU
Q0	20	RR	50 %
Q1	60	RR	30 %
Q2	120	FCFS	20 %

На границе слота обслуживание очереди может приостанавливаться. Неиспользованная часть кванта сохраняется, а текущий процесс остаётся в начале очереди. Для Q2 это сохраняет FCFS внутри очереди до завершения, блокировки либо полного расходования 120 мс CPU. Цена переключения в модель не включена [8]. Календарные доли относятся к очередям при наличии готовой работы. Они не задают гарантированную долю каждого отдельного процесса.

5.2 Результаты воспроизводимого запуска

При seed = 525 завершены все 100 процессов. Модельный интервал составил 28 957 мс, CPU-обслуживание — 28 955 мс, простой — 2 мс. Среднее ожидание равно 17 911,13 мс, среднее оборотное время — 18 519,02

мс. Получены 2047 переключений между разными PID и 397 перемещений между очередями.

Итоговая статистика моделирования представлена на рисунке 5.1.

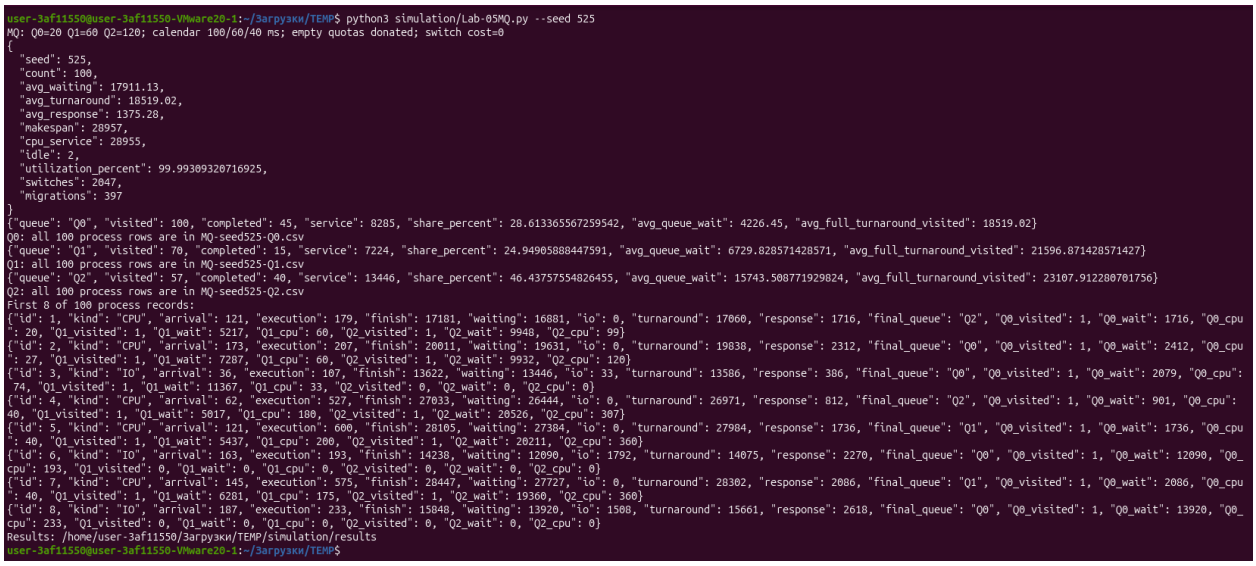


Рисунок 5.1 – Сводная статистика Lab-05MQ

Использование модельного CPU составило 99,9931 %, средний отклик — 1375,28 мс. Показатель использования относится к эмулятору. Баланс времени согласован: 28 955 мс обслуживания и 2 мс простоя дают общий интервал 28 957 мс; оборот процесса включает ожидание готовности, CPU и I/O. Проверка баланса отделяет полезное CPU-обслуживание от времени, когда готовая очередь отсутствует. Статистика использования относится к модельному интервалу.

5.3 Учёт процессов по очередям

Для каждой очереди формируется отдельный CSV со всеми 100 PID. В строке отмечаются посещение очереди, CPU-время и ожидание именно в этой очереди, а также полное оборотное время процесса. Нулевой признак посещения отличает отсутствие процесса в очереди от нулевого ожидания. Сводные показатели приведены в таблице 5.2.

Таблица 5.2 – Учёт времени по очередям

Очередь	Посетили	Завер-шились	CPU мс	Среднее ожидание мс	Средний полный оборот мс
Q0	100	45	8285	4226,45	18519,02
Q1	70	15	7224	6729,83	21596,87
Q2	57	40	13446	15743,51	23107,91

Среднее ожидание очереди рассчитывается по процессам, посетившим её, и включает только ожидание внутри этой очереди. Средний полный оборот рассчитывается по тому же множеству PID и не является временем пребывания только в данной очереди. Один процесс может учитываться среди посетителей нескольких очередей. Средние значения по очередям рассчитаны по разным множествам посетителей. Складывать эти средние для получения полного среднего ожидания некорректно.

Проверка числа записей и пример строк каждой очереди представлены на рисунке 5.2.

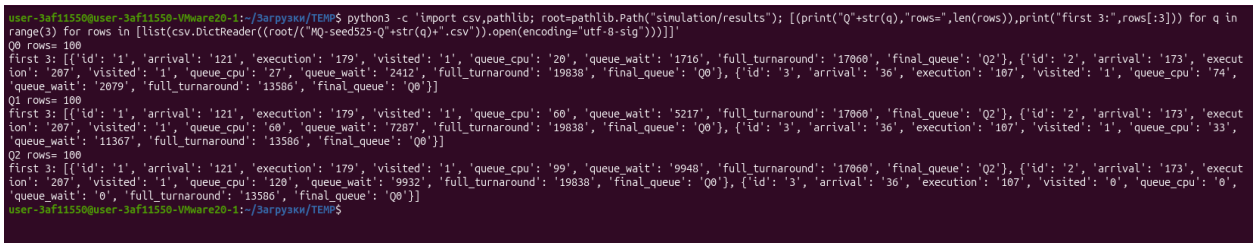


Рисунок 5.2 – Полнота таблиц процессов для Q0 Q1 и Q2

Каждый файл Q0, Q1 и Q2 содержит 100 записей без заголовка. Для PID = 1 представлены arrival = 121 мс, execution = 179 мс и turnaround = 17 060 мс. CPU-времена по очередям равны 20, 60 и 99 мс; их сумма равна 179 мс. Ожидания 1716, 5217 и 9948 мс дают полное ожидание 16 881 мс. PID = 3 не посещал Q2: visited = 0 и её показатели равны нулю. Раздельные столбцы посещения, ожидания и обслуживания позволяют отличить отсутствие процесса в очереди от прохождения через неё без ожидания.

5.4 Анализ переходов и обслуживания

События содержат модельное время, PID, исходную и целевую очередь и причину перехода. Причина quantum соответствует исчерпанию кванта, io-return — возврату после блокировки. Переходы Q1 → Q0 подтверждают повышение после I/O, а переходы Q2 → Q0 и Q2 → Q1 — случайное возвращение CPU-интенсивных процессов на верхние уровни.

Фрагменты журнала с различными причинами переходов приведены на рисунке 5.3.

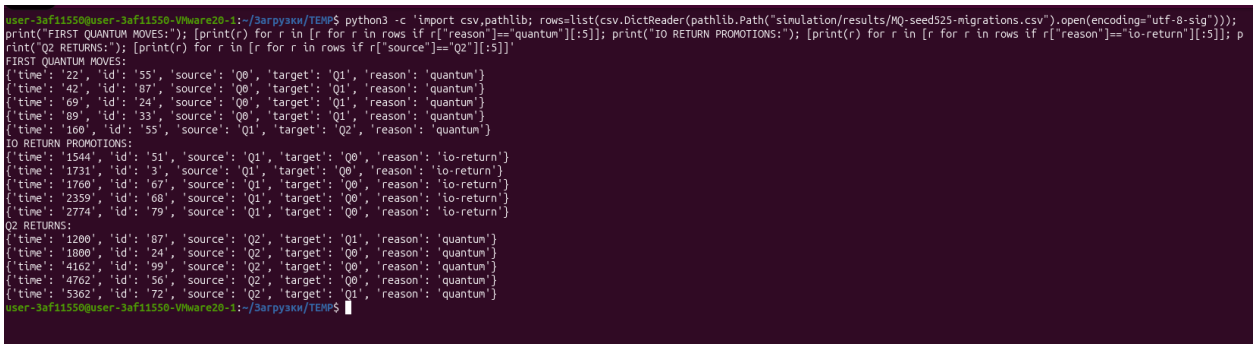


Рисунок 5.3 – Журнал перемещений между очередями

В журнале PID = 55 переведён Q0 → Q1 в момент 22 мс и Q1 → Q2 в момент 160 мс по причине quantum. PID = 51 возвращён Q1 → Q0 после I/O в момент 1544 мс. Для PID = 87 показан переход Q2 → Q1 в момент 1200 мс. Эти записи подтверждают понижение при исчерпании кванта и повышение при возврате. Журнал переходов позволяет проверить применение правил на отдельных событиях. Его записи дополняют итоговые средние, которые не показывают порядок перемещений.

Итоговые доли CPU равны 28,61 % для Q0, 24,95 % для Q1 и 46,44 % для Q2. Календарные бюджеты 50/30/20 действуют при наличии готовой работы во всех очередях; пустые очереди передают свой бюджет другим. Поэтому увеличение доли Q2 за весь опыт согласуется с правилами модели. Выбор Q2 после кванта Q2 не увеличивает счётчик перемещений.

Таким образом, модель MQ позволяет одновременно проследить индивидуальное обслуживание процессов и распределение общей работы между очередями. Совпадение суммарного CPU-времени с исходной потребностью и проверка полного оборота каждого процесса подтверждают согласованность учёта. Отличие фактических долей от календаря объясняется передачей неиспользованных бюджетов готовым очередям.

Заключение

Разработаны и проверены приложения управления приоритетами и родственностью в Windows и Linux. Подтверждены базовые приоритеты Windows 8 / 8 для Normal, 6 / 13 для Below Normal / High и 6 / 10 для Lowest / Highest. В Linux проверены nice 0 / 0 и 19 / −20 по ps и отдельным записям /proc. Полная маска 255 разрешает CPU 0–7; маска 1 ограничивает выполнение CPU 0.

В Windows при первом завершении процессов получены расхождения 1000, 0 и 2000 итераций, для потоков — 0 во всех вариантах. В Linux DELTA = 0 во всех шести опытах. Периодические задержки по 200 мс определяют основную длительность нагрузки, поэтому изменение приоритета не гарантирует различия счётчиков. Полученные единичные результаты не устанавливают устойчивого преимущества процессов, потоков или одной из ОС.

Для десяти заданных задач рассчитаны расписания SJF, RR и CP. Среднее ожидание составило 154,0 для SJF, 170,5 для CP и 207,5 для RR при кванте 80. Среди проверенных квантов 80 даёт минимальное ожидание, небольшие кванты улучшают отклик. Пропускная способность всех трёх алгоритмов равна 0,016 задачи на единицу времени при нулевой цене переключений.

В модели MQ с seed = 525 завершены все 100 процессов; получены 397 перемещений и 2047 переключений между разными PID. Среднее ожидание составило 17 911,13 мс, использование CPU — 99,9931 %. Учёт обслуживания, ожидания и I/O позволил проверить баланс времени и объяснить отличие фактических долей очередей от календарных бюджетов.

Список использованных источников

1 Microsoft Learn. Scheduling Priorities. URL: https://learn.microsoft.com/en-us/windows/win32/procthread/scheduling-priorities (дата обращения: 26.10.2026).

2 Linux man-pages. sched(7) и getpriority(2). URL: https://man7.org/linux/man-pages/man7/sched.7.html (дата обращения: 26.10.2026).

3 Microsoft Learn. GetProcessAffinityMask. URL: https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getprocessaffinitymask (дата обращения: 26.10.2026).

4 Linux man-pages. sched_setaffinity(2). URL: https://man7.org/linux/man-pages/man2/sched_setaffinity.2.html (дата обращения: 26.10.2026).

5 Microsoft Learn. clock. URL: https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/clock (дата обращения: 26.10.2026).

6 Linux man-pages. clock(3). URL: https://man7.org/linux/man-pages/man3/clock.3.html (дата обращения: 26.10.2026).

7 Arpaci-Dusseau R H Arpaci-Dusseau A C. Operating Systems Three Easy Pieces. Scheduling Introduction. URL: https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf (дата обращения: 26.10.2026).

8 Arpaci-Dusseau R H Arpaci-Dusseau A C. Operating Systems Three Easy Pieces. The Multi Level Feedback Queue. URL: https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-mlfq.pdf (дата обращения: 26.10.2026).

Приложение А Структура программ и ключевые операции

Исходные проекты размещены в каталоге Laboratory_05: подкаталог windows содержит решение для Windows, linux — решение и сборочный сценарий для Linux. В каждой среде имеются восемь консольных C++-проектов: Lab-05a, Lab-05x, Lab-05b, Lab-05c, Lab-05SJF, Lab-05RR, Lab-05CP и Lab-05MQ. Общее решение Laboratory_05.slnx объединяет проекты своей среды.

Для Windows дочерние процессы создаются с CREATE_NEW_CONSOLE и CREATE_SUSPENDED. До разрешения общего события устанавливаются класс процесса и маска; основной поток возобновляется ResumeThread. Это исключает выполнение нагрузки с ещё не настроенными параметрами.

В Linux маска задаётся до fork и наследуется дочерними процессами и потоками. nice вызывается в дочернем процессе до exec; потоковая версия вызывает setpriority для текущего потока. Канал запуска блокирует дочерние процессы до готовности родителя.

Файл common/Lab05.hpp содержит операции ОС и общую рабочую нагрузку, common/Scheduler.cpp — модели планирования и сохранение результатов. Интерфейс моделей объявлен в common/Scheduler.hpp. Каждое приложение имеет собственный cpp-файл с точкой входа. Набор десяти задач хранится в tasks.csv. Для сборки в Windows используется build.cmd, для Linux — build.sh; дополнительно предусмотрена сборка через CMake.

После переноса моделей на C++ проверены невытесняющий SJF, немедленное приоритетное вытеснение, граница кванта RR и простой. Расписания SJF, RR и CP для всех исследованных квантов сопоставлены с сохранёнными результатами. Для MQ сверены исходные задачи, временная диаграмма, все строки процессов и журнал перемещений. Инвариант оборот = ожидание + CPU + I/O проверяется для каждого процесса.

Снимки моделирования фиксируют первоначальные контрольные запуски. Подготовленные C++-проекты сохраняют параметры и численные результаты этих опытов; совпадение проверено по полным выгрузкам CSV и JSON, а не только по округлённым средним.

Приложение Б Ответы на вопросы для самоконтроля

1. Планирование и диспетчеризация. Планирование определяет следующего исполнителя и порядок предоставления CPU. Диспетчеризация реализует выбранное решение посредством переключения контекста.

2. Процесс диспетчеризации. Сохраняется контекст текущего потока, выбирается готовый поток, восстанавливается его контекст и при необходимости меняется адресное пространство. После перехода в пользовательский режим продолжается выполнение.

3. Виды планировщиков. Долгосрочный планировщик допускает задания к выполнению; среднесрочный приостанавливает и возвращает процессы; краткосрочный выбирает готовый поток для CPU. Не каждая ОС выделяет эти функции в отдельные компоненты.

4. Степень мультипрограммирования. В классической модели её определяет долгосрочный планировщик посредством допуска новых заданий. Приостановка процессов среднесрочным планировщиком также влияет на число резидентных активных процессов.

5. Алгоритмы планирования. Используются FCFS, SJF, SRTF, RR, приоритетное планирование, многоуровневые очереди и очереди с обратной связью. Для систем реального времени применяются алгоритмы с учётом сроков выполнения.

6. Задачи планирования. Общие задачи — справедливость, использование CPU и отсутствие неограниченного ожидания. Для пакетной обработки важны пропускная способность и оборот; для интерактивной — отклик; для реального времени — соблюдение дедлайнов.

7. Циклическое планирование. Готовые задачи обслуживаются по очереди, каждая получает не более одного кванта за один обход. Незавершённая задача с истёкшим квантом возвращается в очередь.

8. Приоритетное планирование. Из готовых задач выбирается задача наивысшего приоритета. В вытесняющем варианте появление более приоритетной задачи прерывает текущую.

9. Приоритет процесса. Это параметр, влияющий на относительную важность обслуживания. В Windows класс процесса совместно с относительным уровнем потока определяет базовый приоритет; в Linux обычное планирование использует nice отдельных потоков.

10. Базовый приоритет потока Windows. Определяется таблицей сочетаний класса процесса и относительного уровня потока. Диапазон рабочих приоритетов — 1–31; уровень 0 зарезервирован системой. Значения 1–15 относятся к переменному диапазону, 16–31 — к диапазону реального времени. Текущий динамический приоритет может отличаться от базового.

11. Значение nice. Для обычного планирования Linux nice находится в пределах −20…19. Меньшее значение увеличивает вес обслуживания. Самостоятельное повышение приоритета ограничивается правами и лимитами.

12. Кооперативное планирование. Исполнитель сохраняет CPU, пока сам не передаст управление, не заблокируется или не завершится. Уступка исполнителя определяет возможность запуска других задач.

13. Родственное планирование. Планировщик учитывает разрешённые и предпочтительные CPU. Жёсткая родственность ограничивает набор CPU, мягкая предпочтительность может уменьшать миграции и потери кэша без строгого запрета.

14. Маска родственности. Битовая маска разрешённых логических процессоров. В Windows задаётся SetProcessAffinityMask либо SetThreadAffinityMask, в Linux — sched_setaffinity либо pthread_setaffinity_np. Нельзя разрешить CPU вне доступного процессу набора.

15. Перепланировка. Повторное принятие решения о текущем исполнителе. Поводами служат истечение кванта, блокировка, завершение, явная уступка CPU и появление более приоритетного готового исполнителя в вытесняющей системе.